

Backend, then AI on top. September 2026 to April 2027.

Dell SDE II, Java and Spring in production. Study this at the office. Diagrams are written as Mermaid so they stay in the repo even when a slide tool is not open.

PATH · 20 MIN

One profile, not ten careers

You are a Dell SDE II with 3+ years, Java, Spring Boot, microservices, Kafka, async work, Kubernetes/OpenShift, Docker, and already-shipped agent and RAG work. The switch is interview depth on that base, plus Python and AWS practiced on one project. You are not restarting as an AI fresher.

1. Keep the Dell and TCS backend story
2. Add Python only where the agent project needs it
3. Treat Go as reading ability
4. Apply as a backend engineer who ships AI



- Write one sentence you can say in an interview: Senior backend engineer, Java and Spring, distributed systems, with production-style RAG and agents.
- Put FDE in a second folder. Do not make it the January application identity.
- Do not describe Dell proprietary data or internal source in a public repo. Rebuild the shape with synthetic metrics.

Primary: Java
Secondary: Python
Tertiary: Go, enough to read a service and change a handler

Miss. Throwing away three years to become a LangChain beginner.

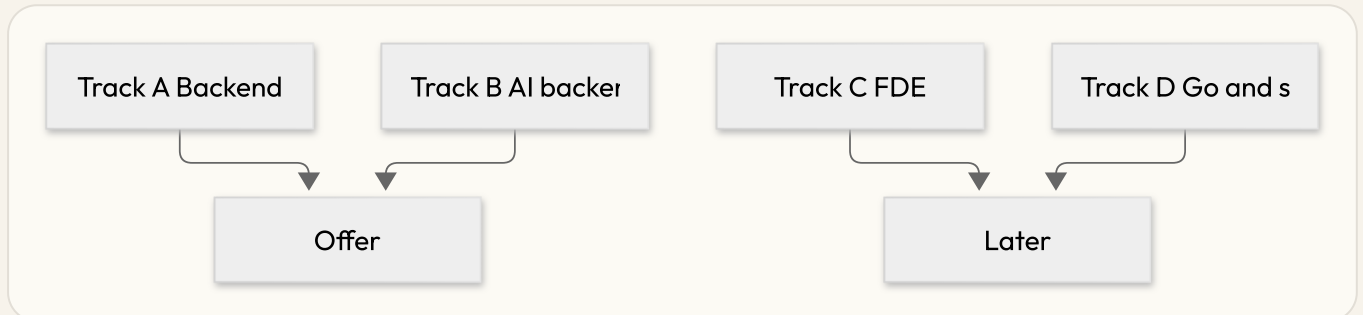
Ask. Walk me through the profile you want in April 2027.

Do. Say the one-sentence profile out loud once. Pin it at the top of the desk.

Four tracks, one weight

Track A is the job you apply for. Track B is the differentiator already started at Dell. Track C is FDE if a posting fits. Track D is Go and storage infrastructure later, which matches object-storage work, not this winter.

1. Backend loop pays the 40-50 target
2. AI work makes the resume specific
3. FDE needs customer discovery
4. Go waits until after the offer



- January applications: backend and AI-platform roles, SDE II / SWE III / senior only where the years match.
- One FDE application is allowed after the flagship README can explain a safe tool call. It is not the whole pipeline.

Atlassian remote-India roles and some India FDE posts exist. US-level pay for India WFH is not the default. Comp follows entity, level, and geo.

Miss. Applying only to FDE titles because the tweets are loud.

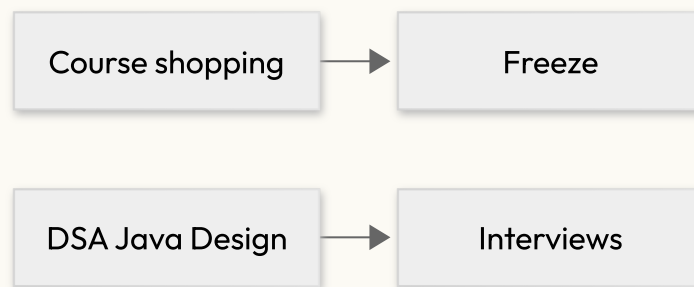
Ask. Which track gets 70 percent of this week?

Do. Mark Track A and Track B as the only open tabs this month.

What stays closed until the offer

The failure mode is a stack of courses and no interviews. React, deep learning research, ten frameworks, CKA plus CKAD together, KMP as a lifestyle, and random hards do not move a Java SDE II loop.

1. Close the extra tabs
2. One project
3. Two credentials later
4. January is for applications



- Skip AWS Cloud Practitioner. It is aimed at people new to cloud.
- Skip AI-900. The current fundamentals exam people mean is AI-901, and even that is optional next to a RAG project.
- Skip LangChain certificates, Python completion badges, and a Docker beginner badge.

Do: AWS Solutions Architect Associate after you can draw a VPC. CKAD after you have deployed the project. AWS Generative AI Developer Professional only after the agents are real.

Miss. Buying the next Udemy bundle because the last one felt unfinished.

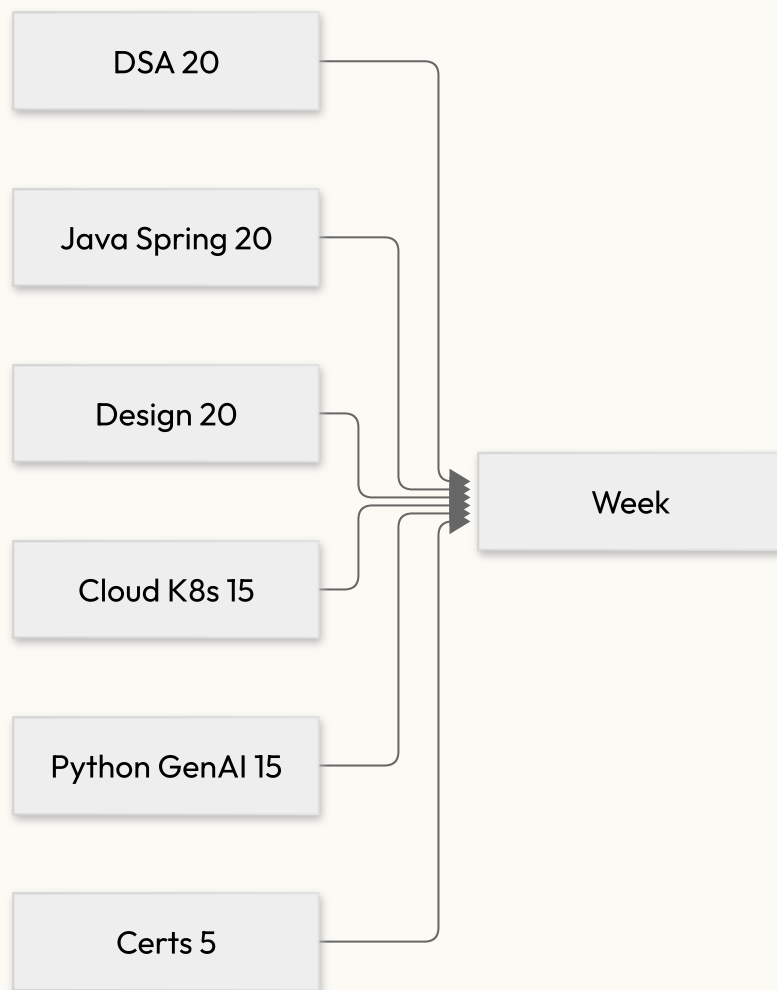
Ask. Name three things you will not study before January.

Do. Write the skip list on paper and leave it in the laptop sleeve.

Where the hours go

About 19 to 20 hours a week. DSA, Java/Spring, and system design take 20 percent each. Cloud and Kubernetes 15. Python and GenAI 15. The project is threaded through those hours, not a fifth full-time job. Certificates are 5 percent.

1. 75 minutes DSA on weekdays
2. 45 minutes backend or design
3. 30 minutes Python, AI, or cloud
4. Saturday is the long build



- Weekday total is 2.5 hours, then stop.
- Saturday: 2 hours DSA, 2 hours design, 2 hours project.
- Sunday: 1.5 hours retrieval, 2 hours project, 1 hour Java or a mock.

A weekday that becomes six hours dies by Wednesday. The cap is the method.

Miss. Adding a night session because the morning felt short.

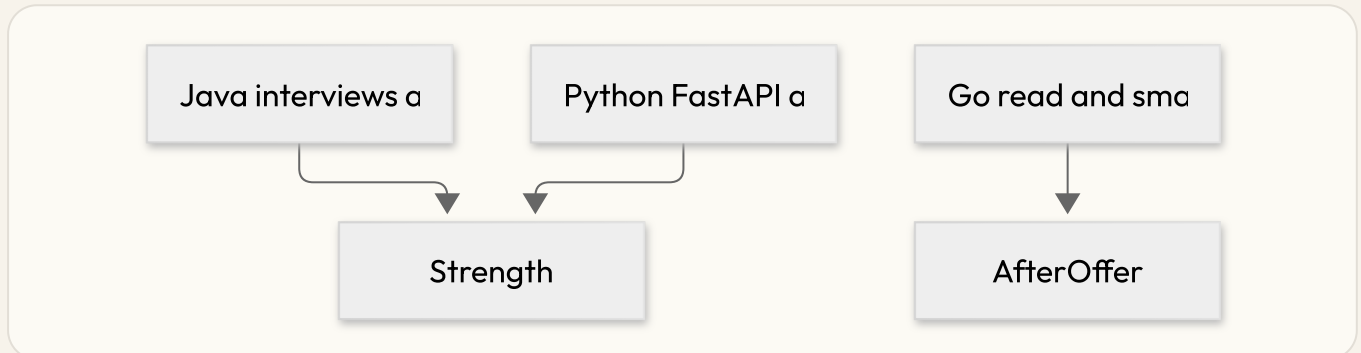
Ask. What do you stop doing at minute 150 on a Wednesday?

Do. Set a repeating timer for 75, 45, and 30. When it rings, stand up.

Java primary, Python secondary, Go later

Postings are polyglot. Google India lists Java, Go, or Python. Apple Bengaluru pairs Spring with Kubernetes, Kafka, and Go or Python. Salesforce and Cisco do the same. Java is not dying in your target set. You already have the production years.

1. Solve DSA in Java
2. Write the agent API in Python
3. Read one Go file a week after December
4. Do not split practice three ways



- Every blank rewrite this month is Java.
- Python starts with the types and tests the flagship service needs, not a 40-hour syntax course.
- Go becomes a weekend reader in 2027: a small HTTP handler, goroutines at a conceptual level, and the storage-service style Apple posts describe.

```
// Java stays the interview language
CompletableFuture<Report> report = diagnostics.collect(bucket);
```

Miss. Being slightly able to start a main method in three languages and fluent in none.

Ask. Which language do you code the medium in?

Do. Open one old PR and explain one CompletableFuture you wrote, without the IDE.

25 Sep to 25 Oct – retrieval and Python syntax

Phase 1 rebuilds the patterns you forget: arrays, hashing, two pointers, sliding window, binary search, linked lists. Java side is collections, HashMap, equality, streams, and concurrency vocabulary. Python is syntax through requests.

1. Identify the pattern before coding
2. Rewrite yesterday with the file empty
3. One Python function, not a course module
4. Stop at 2.5 hours



- Use the Patterns page. Two or three problems per shape.
- Sliding window is one machine: expand right, update state, shrink left, record the answer. Fruit baskets and minimum window are variations.
- HashMap: buckets, equals and hashCode, and what happens on collision. Say it once from memory.

A green mark means you rewrote it alone. A red mark comes back. Day 1, 3, 7, 15, 30.

Miss. Watching the solution and ticking completed.

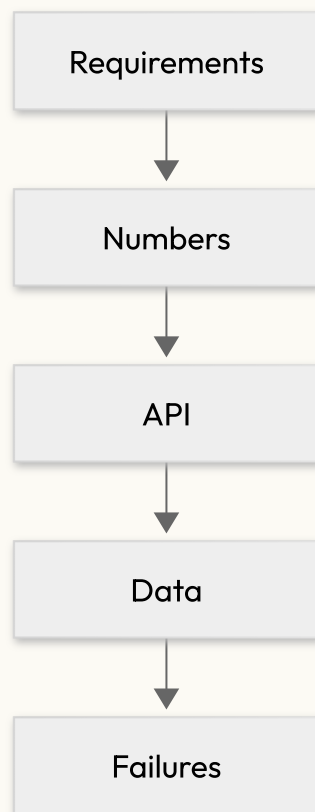
Ask. Is this a new pattern or a variation?

Do. Tonight: Two Sum from a blank file, then say which line updates the map.

26 Oct to 25 Nov – core structures and four designs

Stacks, monotonic stacks, intervals, greedy, trees, BST, heaps. Spring moves from annotations you use to transactions, JPA, Redis, and Kafka failure. Designs: URL shortener, rate limiter, notification service, file storage.

1. One design, 40 minutes, no video first
2. Kafka question is why, not the dependency
3. Tree DFS returns a summary
4. Heap of size K



- Transaction propagation and isolation. Know why a private self-call skips the proxy.
- N+1: one query becomes one per row. Fix with a fetch join or a batch, and say the cost.
- Rate limiter: token bucket, where the counters live, what happens when Redis is down.

Design out loud before drawing. The picture is the notes, not the thinking.

Miss. Starting the video at minute zero.

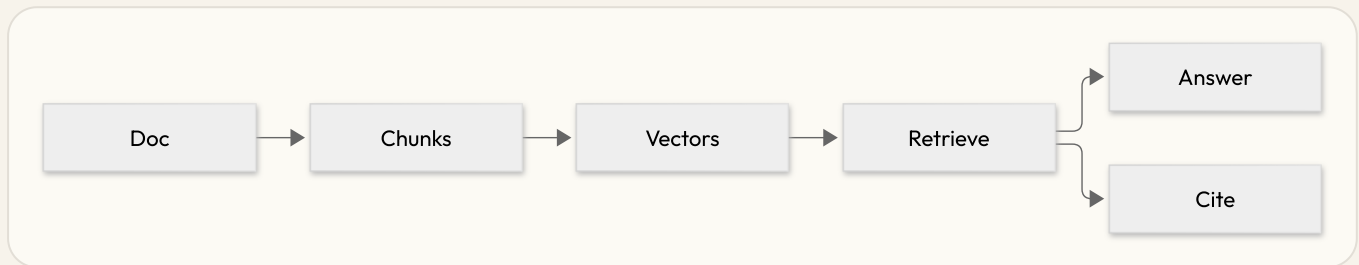
Ask. Where does the rate-limit state live with twenty servers?

Do. Speak a rate limiter for 10 minutes into a voice note. Listen once.

26 Nov to 20 Dec – graphs, cloud, RAG

Graphs through Dijkstra and basic DP. Cloud is IAM, VPC, a real deploy path, Docker, and Kubernetes objects you can debug. AI is tokens, embeddings, a RAG pipeline, and one tool call.

1. BFS is unweighted distance
2. Dijkstra is non-negative weights
3. VPC before EKS
4. RAG cites or it is a demo



- Basic DP only: climbing stairs, house robber, subset sum, coin change, one grid. Advanced MCM waits.
- Deploy the order or diagnostics service in Docker. Then the same image behind a Kubernetes Deployment and Service.
- Python FastAPI endpoint that returns chunks for a question. Keyword overlap is allowed on day one. Embeddings replace it the next week.

January interviews do not require Kosaraju, bridges, or a perfect Z-algorithm. Those stay in the later pile.

Miss. Starting EKS before you can explain a subnet.

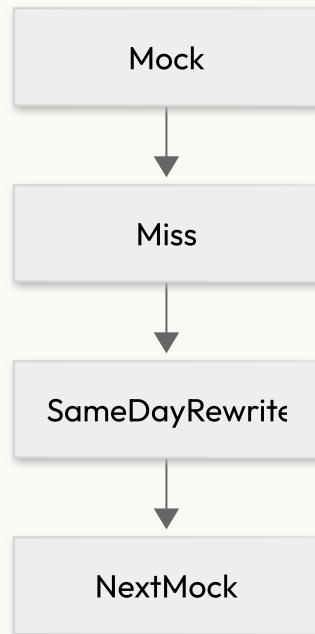
Ask. Why is this graph BFS and not Dijkstra?

Do. Draw your VPC on paper: two subnets, one load balancer, one database not on the public internet.

21 Dec to 15 Jan – mocks, not new syllabi

Mixed retrieval. Five to eight full designs. The flagship README is the project story. Mocks cover Java, Spring, DSA, LLD, HLD, the project, and a behavioral story with a number.

1. No new pattern family
2. Rewrite the miss the same day
3. Project demo in 8 minutes
4. Resume matches what you can draw



- Resume line: backend, distributed systems, AI, cloud. Each bullet has a number you can defend.
- Behavioral set: five STAR stories, 90 seconds, including one agent you shipped and one incident.
- Selective applications start in January. Referrals where you actually know the person.

You joined Dell in April 2026. The one-year mark is April 2027. January is for loops that can land an offer around that window, not for a surprise resignation in October.

Miss. Waiting until you feel 100 percent ready.

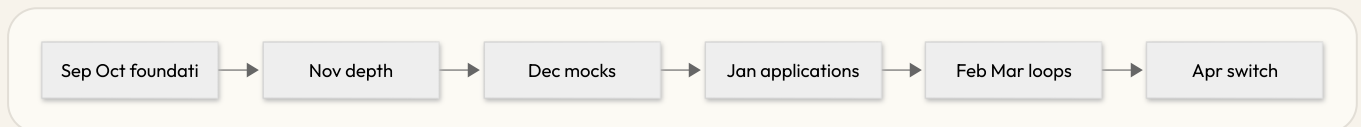
Ask. What did yesterday's mock miss, and did you rewrite it?

Do. Book one mock with a friend or a recording. Listen for the pattern name arriving late.

January apply, April switch

September through October is foundation. November is depth. December is interview shape. January opens the resume, referrals, and selective applications. February and March are loops. April is the switch window after the Dell year.

1. Eight serious applications, not eighty
2. Referral before a cold form when you can
3. 40-50 is a target, not a promise
4. One year at Dell stays intact



- Target list: Walmart SWE III or senior if the years fit, Apple Java/Spring and storage-infrastructure where Go is a plus not a wall, Microsoft 61/62, Atlassian remote-India backend, Adobe, Salesforce, Intuit, Cisco Java/Go roles.
- Compensation on the posting is often hidden. One India FDE post has listed about 35-45 LPA base plus bonus. Product-company medians for strong SDE II sit around your target and move. Negotiate with a number, do not assume a US remote salary.

Foreign company plus India WFH plus US pay is a special case. Ask which entity employs you and how equity is geo-adjusted.

Miss. Mass applying in October with no retrieval habit.

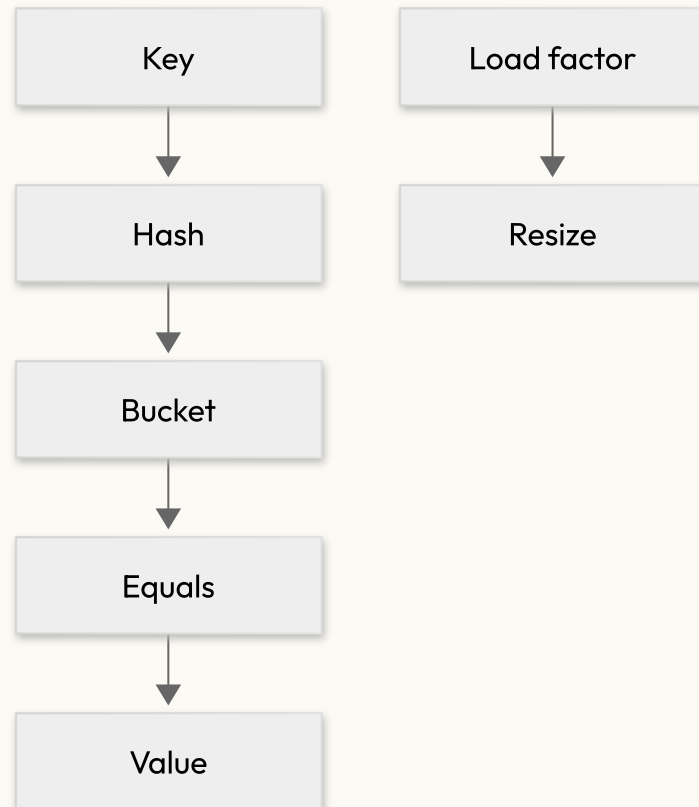
Ask. What is the date you send the first eight applications?

Do. Put 15 January on the calendar as resume freeze, not as a feelings check.

HashMap internals

Interviewers use HashMap to see if you know the structure you call every day. Buckets, hash spreading, equals versus hashCode, and resize are the whole answer.

1. Compute hash
2. Pick a bucket
3. Walk equals
4. Resize when full



- hashCode chooses the bucket. equals decides if the key is the one you want.
- If you override equals you override hashCode. Otherwise a HashSet silently keeps duplicates.
- Java 8 turns a long collision chain into a tree. You should know that the chain is not always a linked list.

```
class Key {  
    final String id;  
    @Override public int hashCode() { return id.hashCode(); }  
    @Override public boolean equals(Object o) {  
        return o instanceof Key k && id.equals(k.id);  
    }  
}
```

Miss. Using the array index as identity and skipping equals.

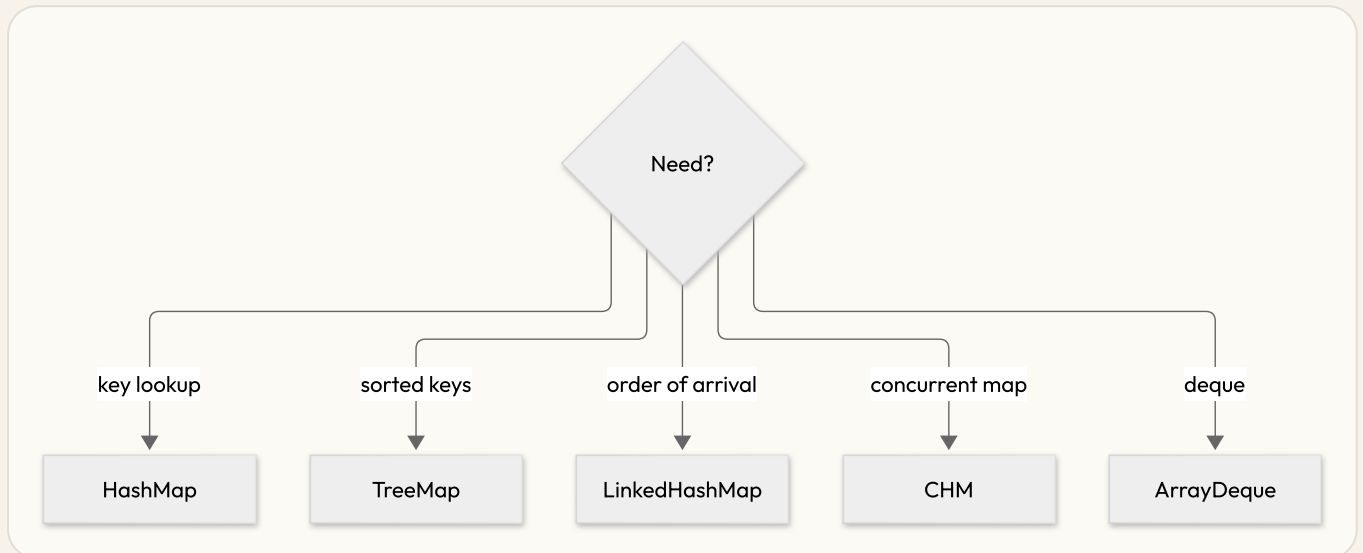
Ask. What breaks if hashCode changes while the key sits in the map?

Do. Break a HashSet on purpose by deleting hashCode. Put the class back.

Collections you must choose out loud

The question is rarely the syntax. It is why this collection, and what it costs.

1. Name the operation
2. Name the cost
3. Reject `java.util.Stack`
4. `ArrayDeque` for stack and queue



- `HashMap` is expected $O(1)$ and unordered. `TreeMap` is $O(\log n)$ and sorted.
- `ConcurrentHashMap` allows concurrent readers and segmented updates. It is not a lock around the whole map.
- `Optional` is for a return value that might be absent. It is not a field type on an entity.

`Stack` is a legacy synchronized `Vector`. Say `ArrayDeque`.

Miss. Picking a `List` and scanning it when a map was the point.

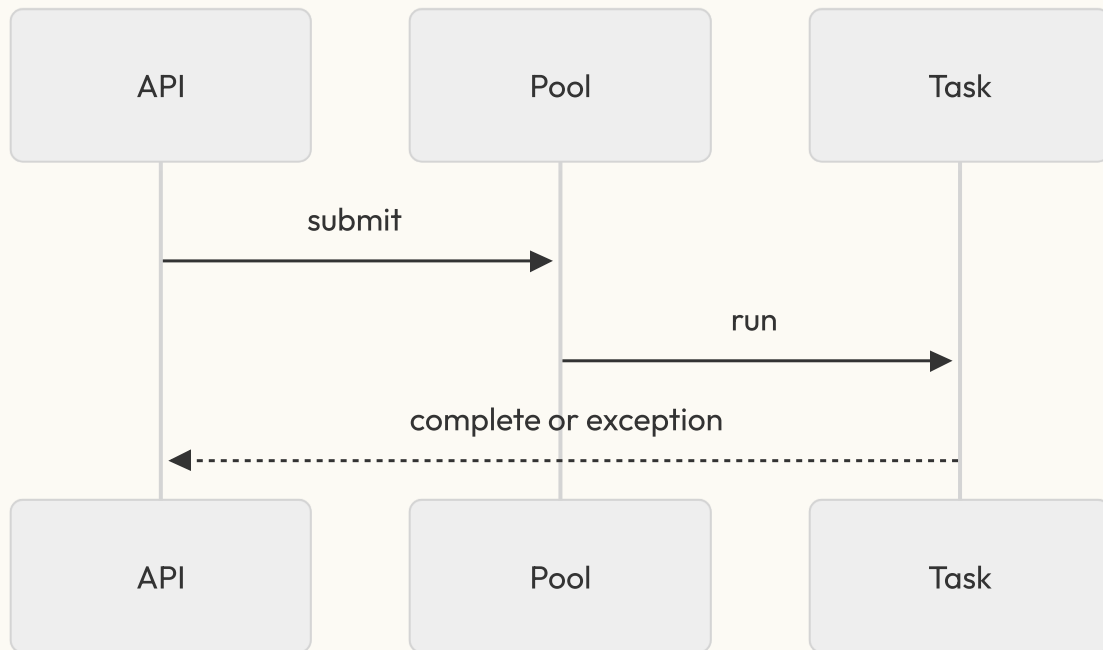
Ask. When would you refuse a `HashMap`?

Do. Write three method signatures and name the collection in the margin before any code.

Executors, futures, and CompletableFuture

Your resume already mentions asynchronous work. The interview will ask what runs where, what happens on failure, and how you avoid blocking the common pool.

1. Do not new Thread in a request
2. Name the pool
3. Handle the exception
4. Do not block on the common pool



- `ExecutorService` owns the threads. `Future` is the handle. `CompletableFuture` composes them.
- `supplyAsync` on the common pool is fine for CPU work that is short. Blocking IO needs your own pool.
- `exceptionally` or `handle`, or the exception disappears until someone joins.

```
CompletableFuture<String> body = CompletableFuture
    .supplyAsync(this::readLogs, ioPool)
    .exceptionally(error -> "logs unavailable");
```

Miss. Calling `get()` on the thread that is supposed to stay free.

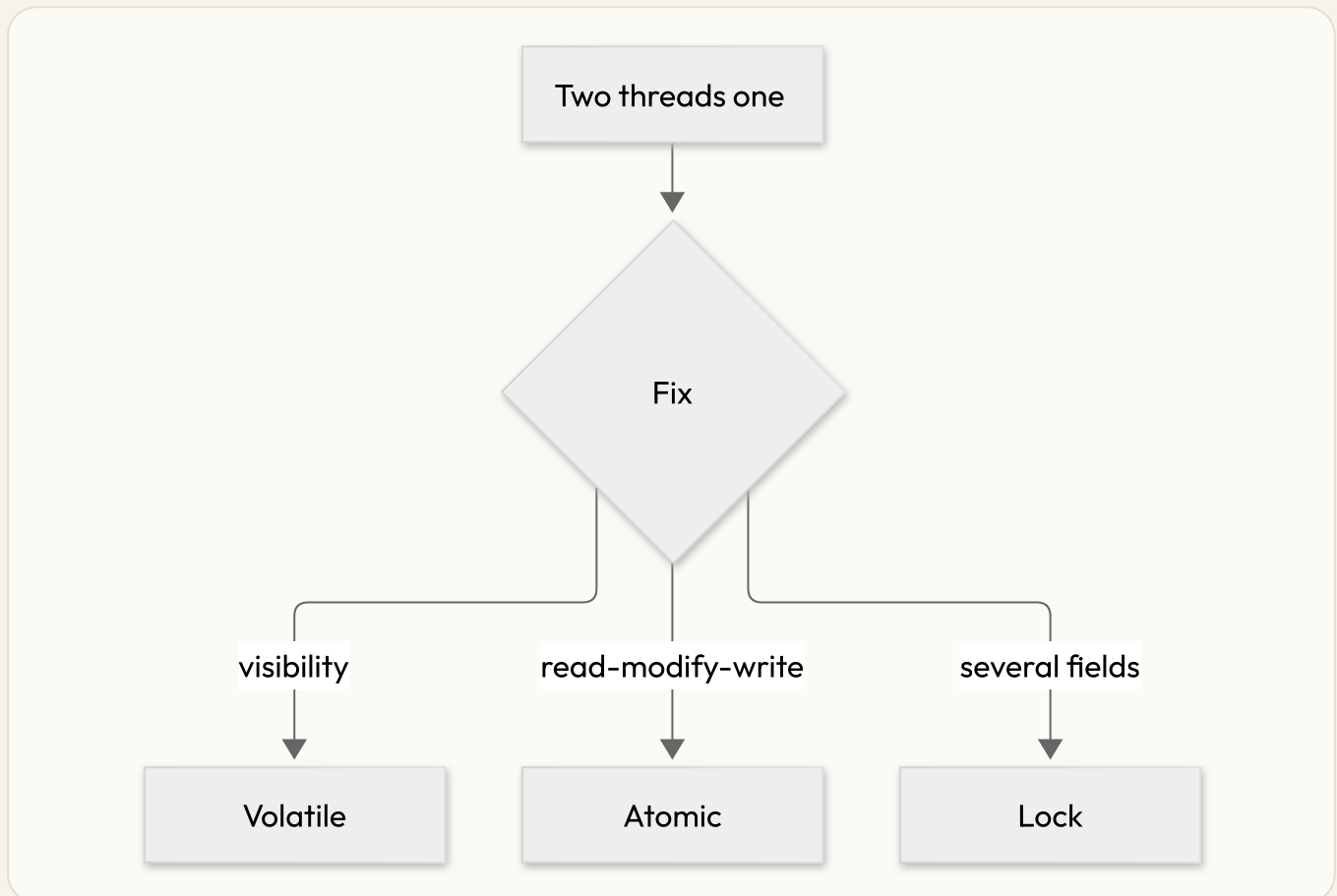
Ask. What happens if the callable throws?

Do. Draw the pool you would use for log collection versus the pool for a CPU score.

volatile, locks, atomics, deadlock

You need the vocabulary of visibility and exclusion. You do not need to recite the Java Memory Model chapter.

1. volatile is visibility
2. Atomic is a single variable update
3. Lock guards a block
4. Deadlock is lock order



- volatile does not make `count++` safe. The increment is three steps.
- `synchronized` or `ReentrantLock` gives exclusion. Always take locks in the same order.
- A deadlock story: thread A holds lock 1 and wants lock 2, thread B holds lock 2 and wants lock 1.

`AtomicLong` for a counter. A lock when two fields must change together.

Miss. Sprinkling volatile on a compound action and calling it thread-safe.

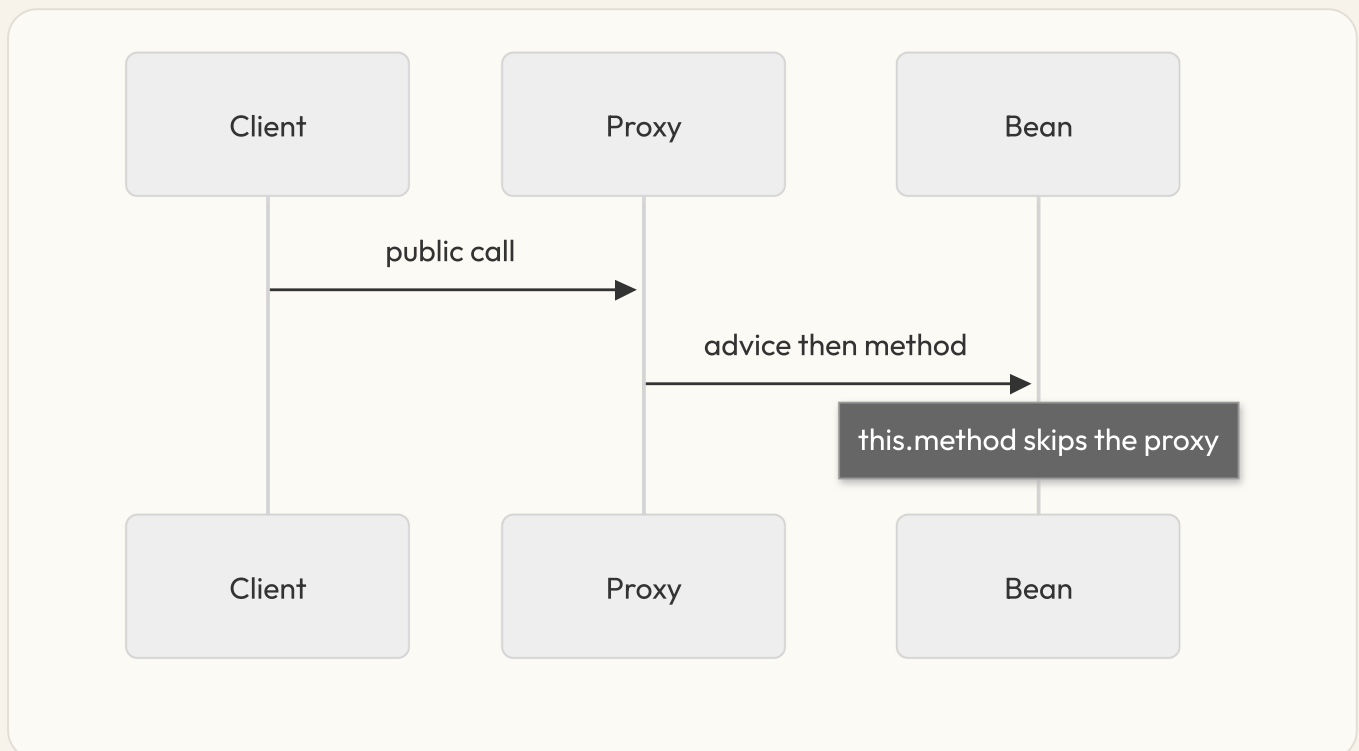
Ask. Give a race that volatile does not fix.

Do. Write a 6-line deadlock on paper, then the lock-order fix.

IoC, beans, and the proxy

Spring creates the objects and injects them. Transactions, security, and async work through proxies. If you call a proxied method from inside the same class, the proxy is skipped.

1. Container builds the bean
2. Proxy wraps cross-cutting work
3. Self-invocation skips it
4. Constructor injection is the default



- Prefer constructor injection. The bean is immutable and tests can pass fakes.
- Bean lifecycle in one breath: construct, inject, aware callbacks, init, use, destroy.
- Auto-configuration is conditional beans. You override them by declaring your own.

`@Transactional` on a private method or a self-call does nothing useful.

Miss. Field injection because it is fewer lines.

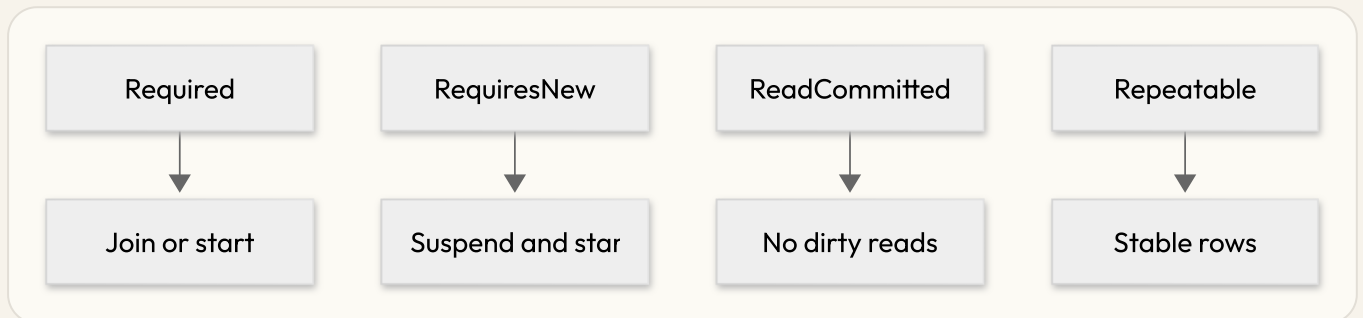
Ask. Why did this `@Transactional` not roll back?

Do. Point at one service in your Dell work and say which calls go through the proxy.

Transactions, propagation, isolation

This is SDE II territory. You should say what is atomic, what isolation you need, and what a nested call does.

1. Name the boundary
2. Name propagation
3. Name isolation
4. Say what rolls back



- REQUIRED joins the current transaction. REQUIRES_NEW commits the inner work even if the outer rolls back. That is how you persist an audit row.
- Default rollback is runtime exceptions. Checked exceptions need rollbackFor.
- Isolation: read committed is the usual Postgres setting. Repeatable read stops a row changing under you. Phantom rows are a different problem.

The outbox insert and the order insert share one transaction. The Kafka send does not.

Miss. A long transaction that calls an external API.

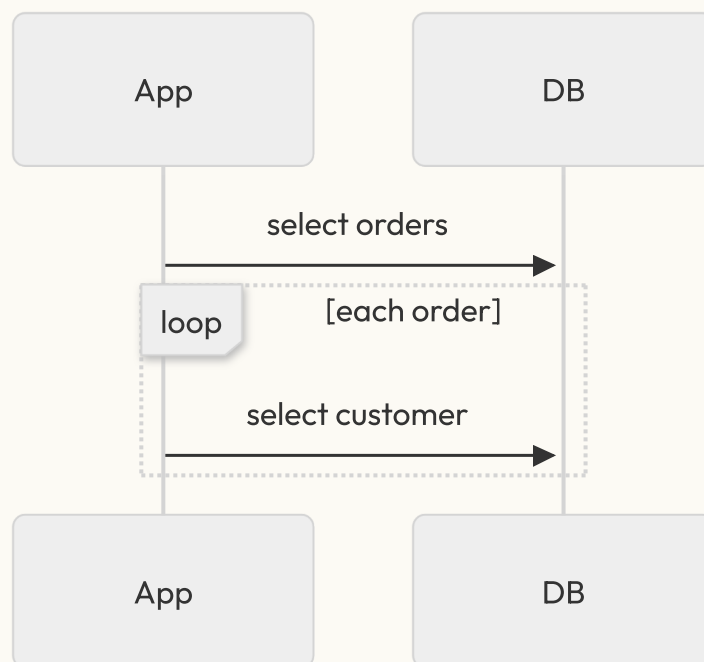
Ask. When do you want REQUIRES_NEW?

Do. Write the two methods for order plus outbox and mark which transaction they share.

JPA, N+1, locking

Hibernate will happily issue a query per row. You should notice it before production does.

1. One query becomes N
2. Fetch join or batch
3. Lazy is a loaded graph
4. Version column for optimistic lock



- N+1 shows up when you touch a lazy association in a loop.
- Optimistic lock: a version column, conflict becomes an exception, the client retries. Pessimistic lock holds a database lock. Use it rarely.
- Eager everything is not the fix. You load the graph you need for this request.

`@Version` on the aggregate that two agents might update.

Miss. Open-session-in-view hiding the N+1 until a loader disappears.

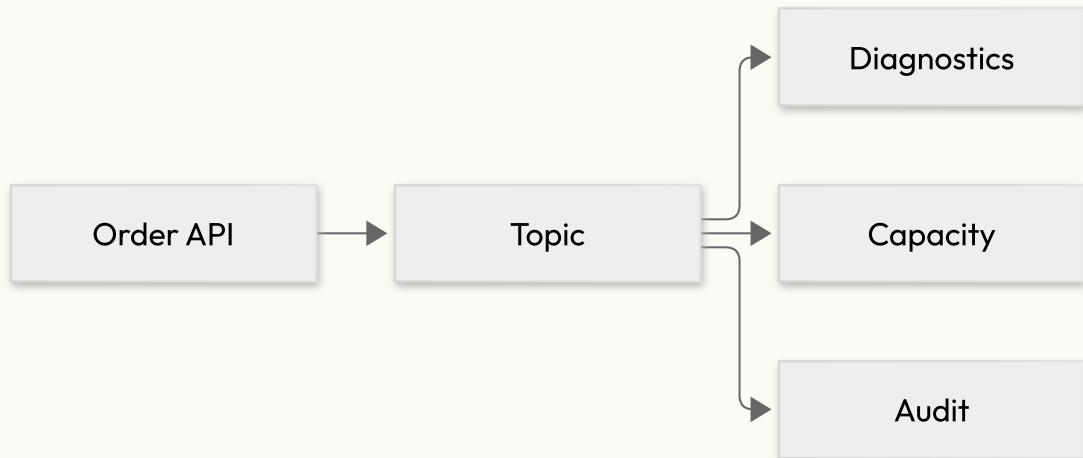
Ask. How do you detect N+1 locally?

Do. Turn on SQL logging for one repository test and count the selects.

Why Kafka, not another REST call

REST couples the caller to the callee being up. A log of events lets the diagnostics agent, the capacity agent, and the audit trail each consume at their own speed.

1. Producer commits the fact
2. Consumers are independent
3. Replay is possible
4. REST was the wrong coupling



- Use Kafka when more than one downstream cares, when you need a buffer, or when you must replay.
- Do not use Kafka for a request that needs the answer in the same HTTP call.
- Ordering is per partition key. Choose the key on purpose, such as bucket id.

Key = bucketId so one bucket's events stay ordered.

Miss. A topic with a random key and a later complaint that events arrived out of order.

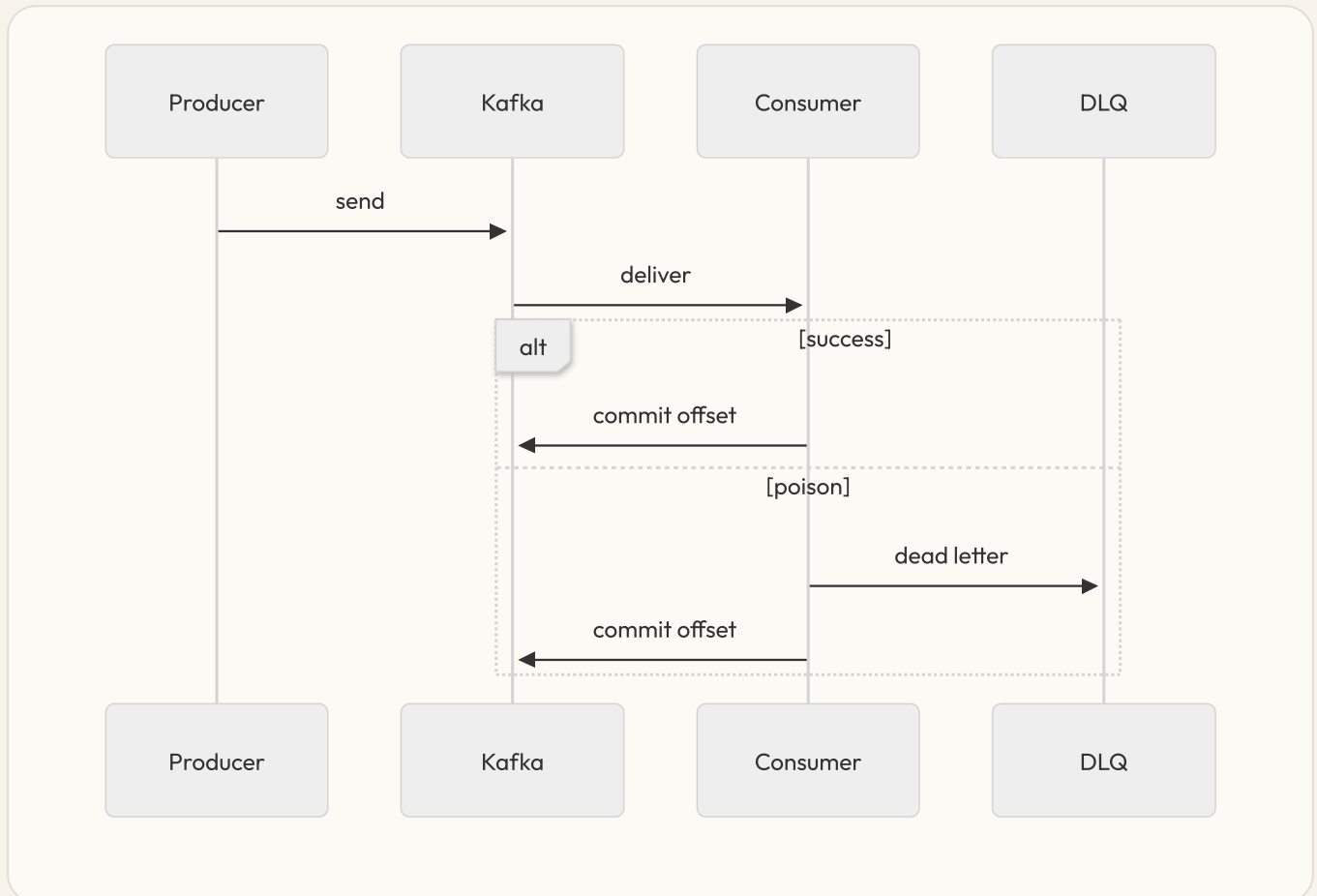
Ask. Why not just call the agent over HTTP?

Do. Name one Dell flow that is a fact other systems observe, and one that must stay a request.

Consumer crash, duplicates, retries, DLQ

This is the senior part of your Kafka story. Delivery, crashes, and duplicates are one design.

1. At least once is the default
2. Crash before commit redelivers
3. Handler is idempotent
4. Poison goes to DLQ



- If the process dies after the side effect and before the offset commit, the message comes back. Your handler must tolerate that.
- Idempotency key on the event. The second delivery finds the row and returns.
- Retries with backoff for a down dependency. A DLQ when the payload itself is bad. Do not retry forever.

Store processed event ids. On duplicate, ack and skip.

Miss. Committing the offset before the work, then losing the crash window's events.

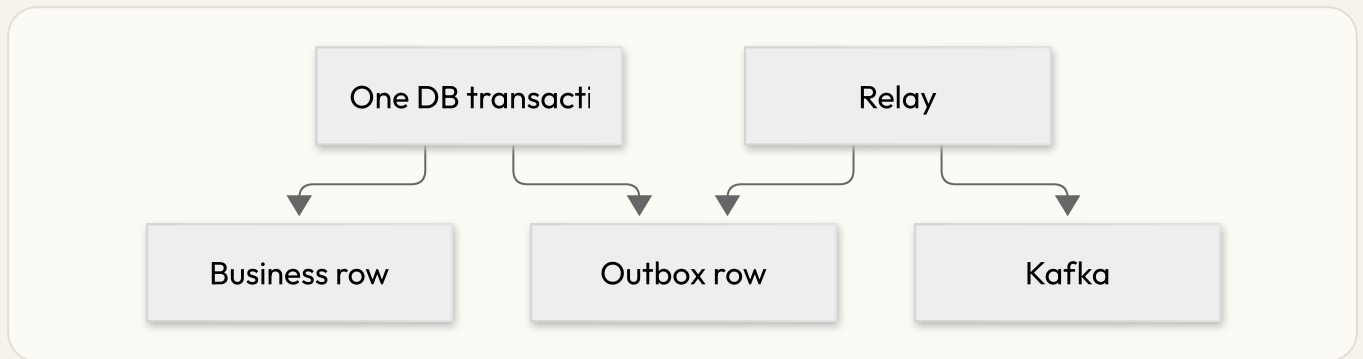
Ask. The consumer dies after writing the database and before commit. What do you do?

Do. Add an idempotency key to the project event on paper. Three lines.

Outbox

You cannot commit a database row and a Kafka send as one transaction without a pattern. The outbox is that pattern.

1. Same transaction
2. Relay reads the outbox
3. Mark sent
4. Kafka is outside the transaction



- Insert the business row and the outbox row together.
- A worker publishes and then marks the outbox sent. If it crashes, it publishes again. Consumers stay idempotent.
- This is the story that connects your order API and your agents.

```
outbox(event_id, type, payload, sent_at)
```

Miss. Publishing to Kafka inside the transaction and hoping both commit.

Ask. What is atomic in the outbox pattern, and what is not?

Do. Sketch the table and the worker loop in the flagship README.

Redis as cache, lock, and agent memory

Redis is the fast state: rate-limit counters, a cache in front of bucket metadata, and a short-lived agent session. It is not your source of truth.

1. Cache has a TTL
2. Source of truth stays in Postgres
3. Stampede is a real bug
4. Session state expires



- Cache aside: read cache, on miss read the database, fill the cache.
- Set a TTL. Invalidate on write when stale data would be wrong, such as a quota.
- A single-flight lock or a short lease stops a hundred misses from hammering Postgres.

```
SET bucket:42 meta EX 30
```

Miss. Caching a permission decision with no TTL.

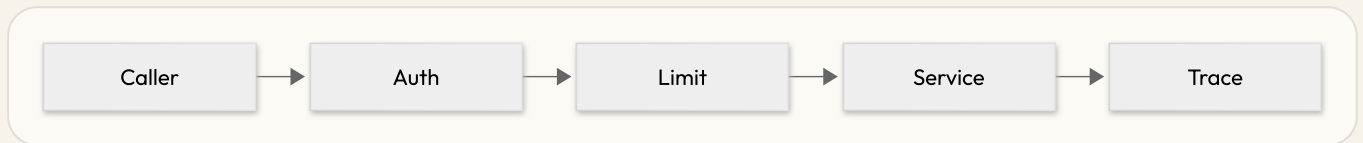
Ask. What is wrong if Redis is empty after a restart?

Do. Name three keys the flagship project would store, and the TTL for each.

Security, resilience, actuator

JWT and OAuth answer who is calling. Resilience answers what you do when the next service is sick. Actuator and traces answer what happened.

1. Authenticate then authorize
2. Timeout every remote call
3. Retry only idempotent calls
4. A trace id crosses the agent



- Spring Security: filter authenticates, method or route rules authorize.
- Resilience4j or a simple policy: timeout, small retry with jitter, circuit breaker when the dependency is down.
- Micrometer plus actuator. A correlation id from the HTTP request lands in the Kafka header and the agent log.

Do not retry a payment POST unless the idempotency key makes the retry safe.

Miss. Logging the full prompt and the customer's object listing.

Ask. Which calls are safe to retry?

Do. Add one correlation id to the story you will tell about DefectIQ-style RAG. No internal data.

How every design is told

SDE II loops at Amazon and Microsoft put HLD next to DSA. You use the same spine every time so you do not freeze on the noun they picked.

1. Clarify read and write
2. One number for scale
3. API then data
4. Failure before you stop

Requirements



Scale



API



Data model



Architecture



DB



Cache



Queue



Consistency



Failure



Observe



Security



Tradeoffs

- Functional requirements in five bullets. Non-functional: latency, availability, consistency.
- Back-of-envelope: QPS, storage per day, read/write ratio. Order of magnitude is enough.
- End on trade-offs. A design with no trade-off is a drawing.

40 minutes. Talk first. Draw second. No video until you have spoken.

Miss. Jumping to Kafka because it sounds senior.

Ask. What did you refuse to build in v1?

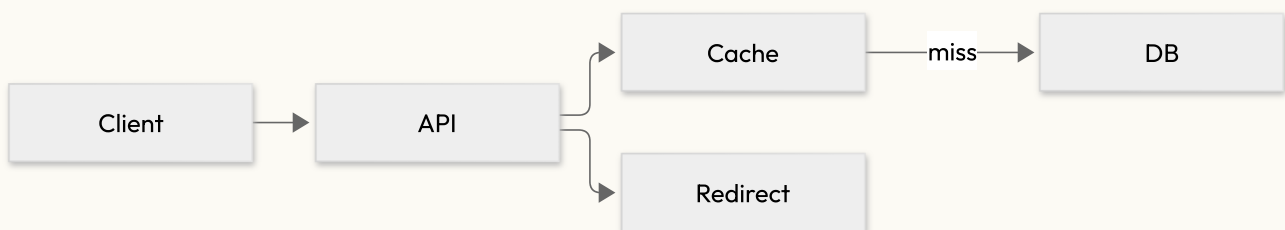
Do. Run the spine on a system you already operate. Eight minutes.

DESIGN · 40 MIN

URL shortener

The classic is a key-value write with a read-heavy cache. It teaches ids, redirects, and cache.

1. Generate an unguessable id
2. Store url by id
3. Cache the hot ids
4. 301 versus 302



- Write path: hash or an id service, unique constraint, return the short url.
- Read path: cache then database. Redirect. Do not count analytics on the request thread.
- Analytics is an async event. That is your first queue in this design.

GET /{id} -> 302 Location

Miss. A sequential integer id that leaks volume and is easy to scrape.

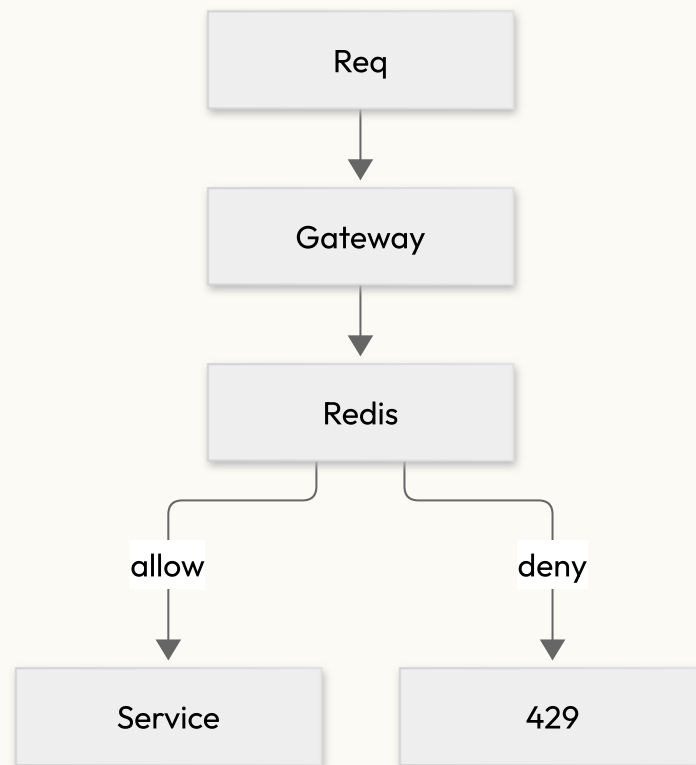
Ask. How do you keep one id from being issued twice?

Do. Speak it once with a cache and an analytics event.

Rate limiter

Apple and backend loops ask this because it is a small distributed system: counters, clocks, and failure.

1. Token bucket
2. Key is the client
3. Redis is shared
4. Fail open or closed on purpose



- Token bucket: capacity and refill rate. Each request takes a token.
- Key by user or token, not one global bucket, unless the limit is for the whole cluster.
- If Redis is down, fail open for a product page and fail closed for a write that spends money. Say which.

`INCR + EXPIRE` is a crude fixed window. Mention the boundary burst.

Miss. A limiter in one process memory when you have twenty pods.

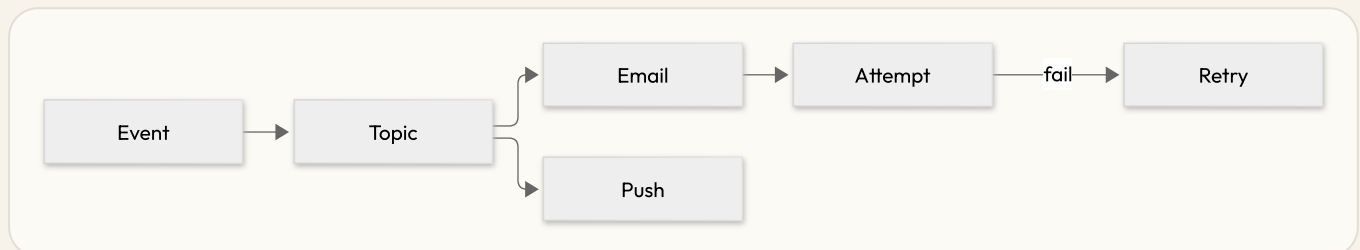
Ask. Two requests at the same millisecond. What happens?

Do. Put this in front of the agent tool-execution endpoint. That is the same design.

Notification system

Fanout, retries, and idempotent delivery. This is the Kafka design with a user-visible result.

1. Event in
2. One worker per channel
3. Retry with backoff
4. Do not double-send



- The API accepts the intent and returns. Delivery is async.
- Idempotency key so a retry does not email twice.
- Preferences and quiet hours are a filter before the provider call.

`notification_id` unique at the provider adapter.

Miss. Calling the email vendor inside the user request.

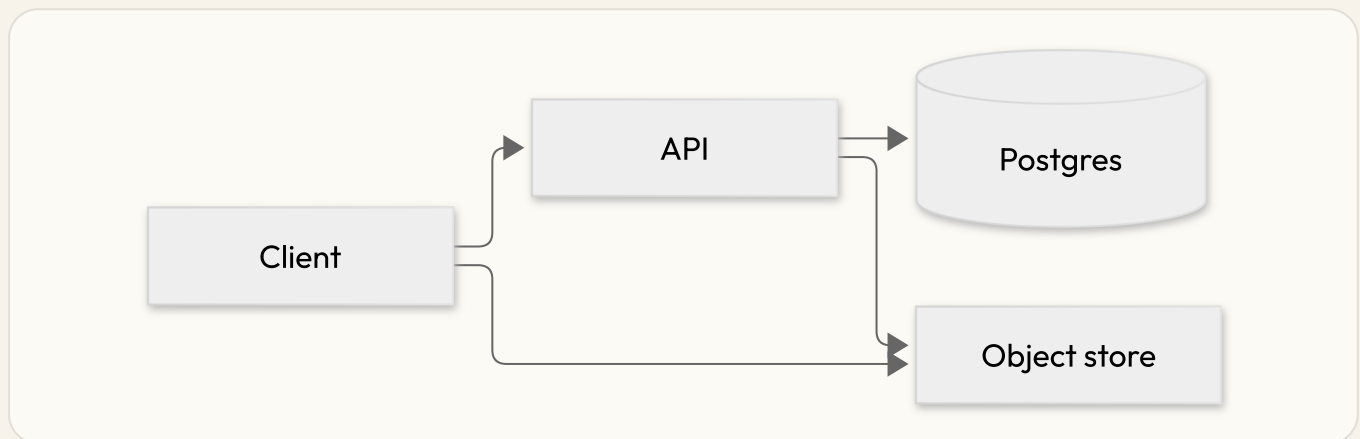
Ask. The worker crashes after the vendor accepts. Did the user get two mails?

Do. Map this onto an incident notice for a bucket, with synthetic users.

File and object storage

This is the design closest to your storage background. Metadata in a database, bytes in object storage, upload in parts.

1. Metadata is not the bytes
2. Multipart upload
3. Checksum
4. Lifecycle



- The API issues an upload plan. The client sends parts to object storage.
- Complete is a metadata transaction: all parts present, checksum matches, object becomes visible.
- A garbage collector deletes abandoned parts. Say the delay.

HEAD for existence. GET via a short-lived signed URL.

Miss. Storing the file bytes in Postgres.

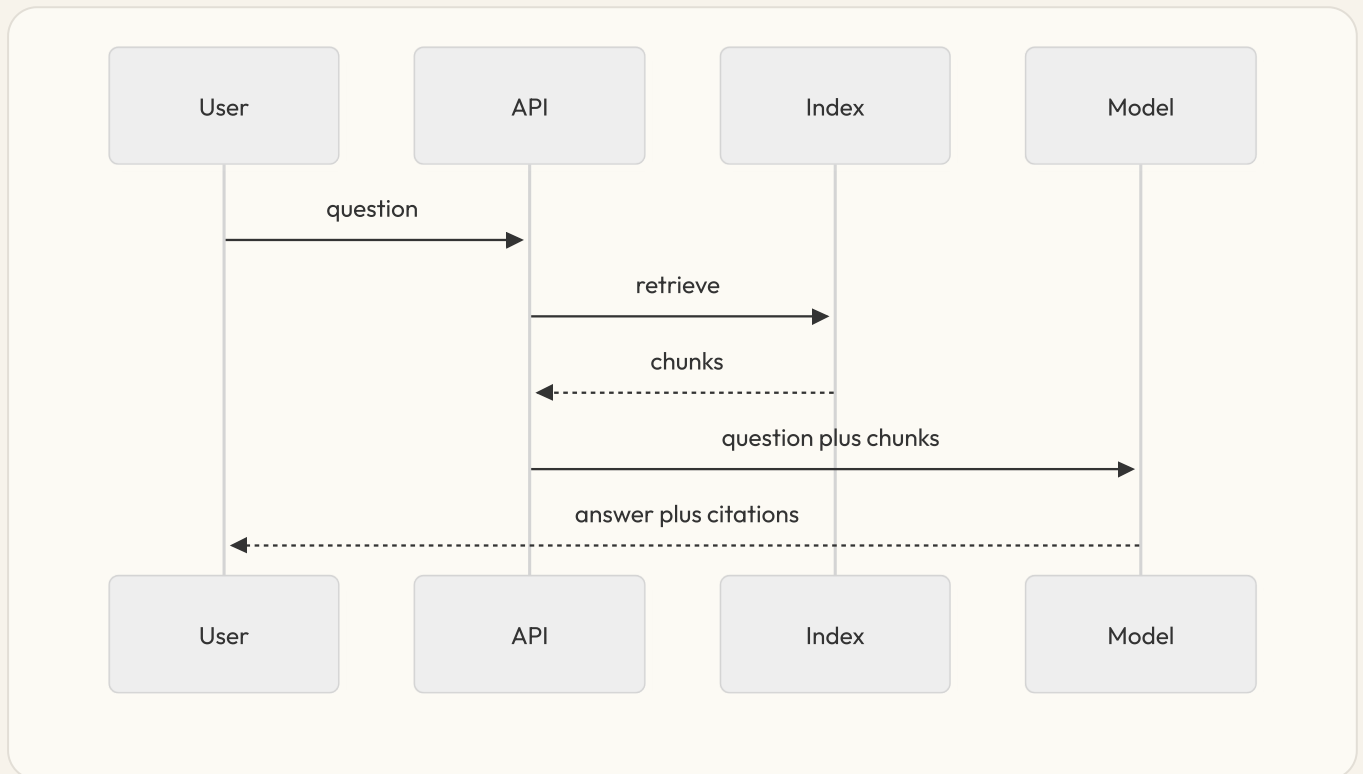
Ask. A part never arrives. What does the user see?

Do. Tell this design with a bucket you invent. No Dell hostnames or schemas.

AI chat with RAG

The design interview version of your RAG work. Retrieval, citations, and what you do when the model is wrong.

1. Ingest offline
2. Retrieve before generate
3. Cite the chunk
4. Evaluate a fixed set



- Ingestion is a pipeline, not part of the user request.
- The prompt may only use retrieved text for factual claims. If retrieval is empty, say so.
- A set of questions with expected facts. Track pass rate, latency, and cost per answer.

Answer: chunk 14 says the quota is 80 percent. Link the chunk.

Miss. A chatbot that answers from memory with no source.

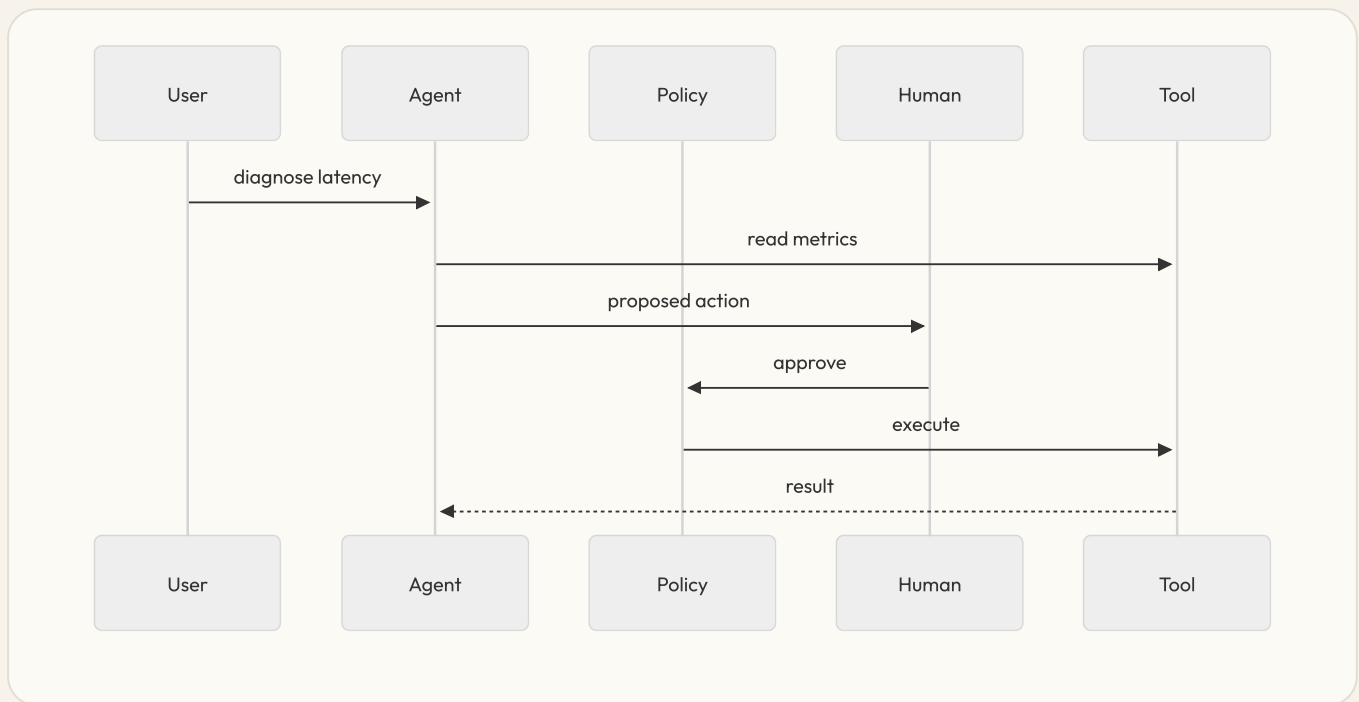
Ask. How do you know the assistant is not inventing a root cause?

Do. Write 5 evaluation questions for a fake storage runbook.

Agent orchestration with a human gate

The differentiator. An agent may suggest a change. It may not apply the change until a person approves, the tool is allow-listed, and the call is idempotent.

1. Read tools are wider
2. Write tools need approval
3. Timeouts and retries
4. Trace every tool call



- Split tools into read and write. Diagnostics are read. A config change is write.
- The approval record stores who, what, and the idempotency key.
- Prompt injection in a log line must not be able to call the write tool. The model does not hold the credential. The policy service does.

Tool list: getMetrics, getLogs, proposeScale. Only proposeScale needs a human.

Miss. Giving the model a kubeconfig.

Ask. How do you stop an agent from looping on a tool?

Do. Draw the approval box on the flagship diagram before you write more code.

Cache, chat, and feed in one sitting

Three designs share a spine. Learn the part that makes each one different, not three new religions.

1. Cache is freshness
2. Chat is a channel per conversation
3. Feed is fanout on write or read
4. All three need a queue

Cache: TTL and s

Chat: fanout and

Feed: push versu:

- Distributed cache: cache aside, TTL, single-flight on miss. Redis in front of Postgres.
- Chat: a connection tier, a message log, fanout to online users, store for history. Presence is ephemeral.
- Feed: push to followers' inboxes when celebrities are rare. Pull or a hybrid when one account has millions of followers.

The hard part of chat is not the bubble. It is ordering in a conversation and not losing a message on reconnect.

Miss. Designing all three as a single database table.

Ask. When does feed fanout on write fall over?

Do. Pick feed. Say push, pull, or hybrid in one sentence and why.

Search, payments, scheduler, logs

Four more nouns. Each has one trick. Say the trick first.

1. Index asynchronously
2. Charge once
3. Only one worker owns a job
4. Logs are append-only

Search: index is n

Pay: idempotent

Scheduler: lease

Logs: buffer then

- Search: the database is the source. An index (inverted or a service) is built from a stream. Queries hit the index.
- Payments: idempotency key, ledger entries that sum, and a state machine. Never a single balance update with no history.
- Scheduler: a job row with a lease timestamp. A worker updates the lease. If it dies, another worker takes the expired lease.
- Log aggregation: agents buffer, a queue absorbs bursts, object storage keeps the bulk, an index keeps the recent window.

Payment states: created, authorized, captured, failed. Illegal jumps are rejected.

Miss. A cron on every pod so the job runs five times.

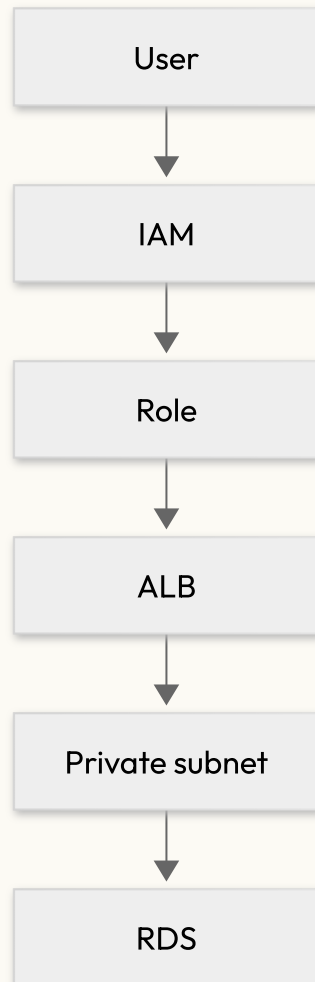
Ask. Two API calls with the same idempotency key. How many charges?

Do. Write the payment states on a card. Carry it to one mock.

IAM and VPC before any service

Solutions Architect Associate is the credential, later. The knowledge starts with who can call what, and which network path exists. Cloud Practitioner is the wrong exam for you.

1. No long-lived keys on laptops
2. Roles for services
3. Database is private
4. Security groups are stateful



- IAM user is a person. A role is what EC2, EKS, or Lambda assumes. Policies are the allow list.
- VPC, public subnet for the load balancer, private subnet for the app and the database.
- Security group on the database allows the app group on 5432 and nothing from the internet.

SAA is the supporting certificate after you can draw this. Do not start with Cloud Practitioner.

Miss. Opening port 22 to the world because a tutorial did.

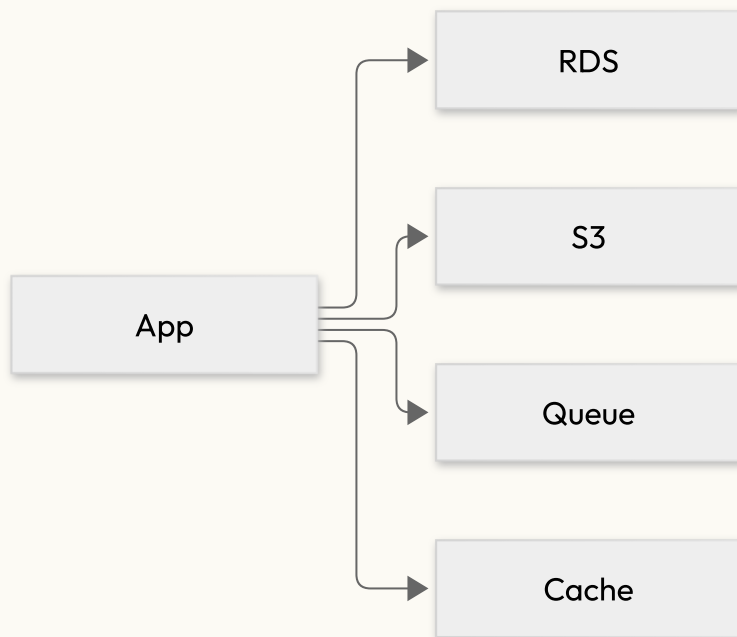
Ask. How does the pod reach S3 without an access key in the image?

Do. Draw the VPC once. Photograph it. That is this week's cloud note.

S3, RDS, queues, and cache

The flagship maps cleanly: bytes in S3, metadata in RDS, events on SQS or Kafka, hot state in ElastiCache.

1. S3 is objects
2. RDS is Postgres
3. SQS is a buffer
4. ElastiCache is Redis



- S3: bucket, key, storage class, lifecycle, pre-signed URL. Not a filesystem you mount and forget.
- RDS Postgres for the source of truth. Multi-AZ is availability, not a write-scaling strategy.
- SQS for a simple buffer. Kafka when you need replay and several consumers. Say the difference.
- ElastiCache Redis for the rate limit and the cache. Same rules as the Redis topic.

A pre-signed URL expires. The client uploads directly. Your API stays small.

Miss. Using SQS and also expecting to rewind the stream a week later.

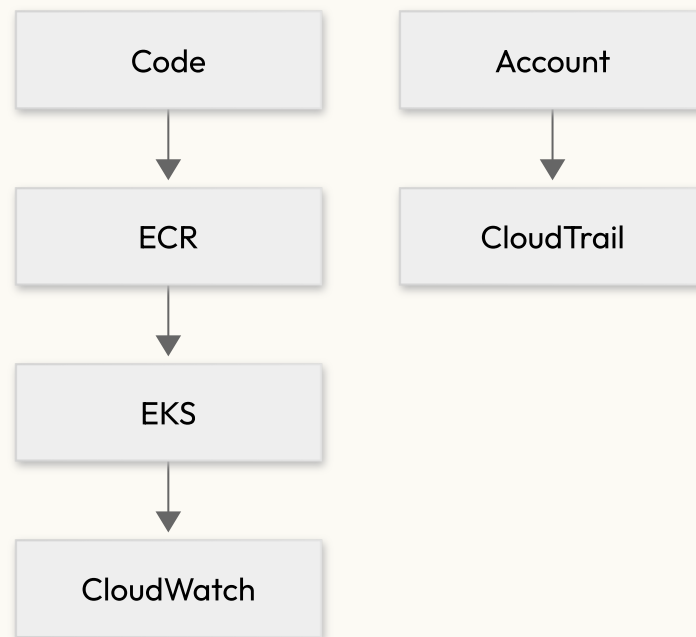
Ask. When do you pick SQS over Kafka?

Do. Label the four boxes on the flagship diagram with the AWS name.

EKS, images, and CloudWatch

EKS is Kubernetes with AWS as the cloud under it. You still need to understand pods. CloudWatch and CloudTrail tell you what the service and the account did.

1. Build an image
2. Push to ECR
3. Deploy to EKS
4. Logs and a trail



- ECR stores the image. EKS runs it. IAM on the service account is how the pod calls AWS.
- CloudWatch for logs and metrics. An alarm on error rate and on disk.
- CloudTrail is the audit of API calls. Useful when someone changes a security group.

You do not need Lambda, API Gateway, and Route53 in week one. Add them when the design needs them.

Miss. Putting the AWS key in the container environment.

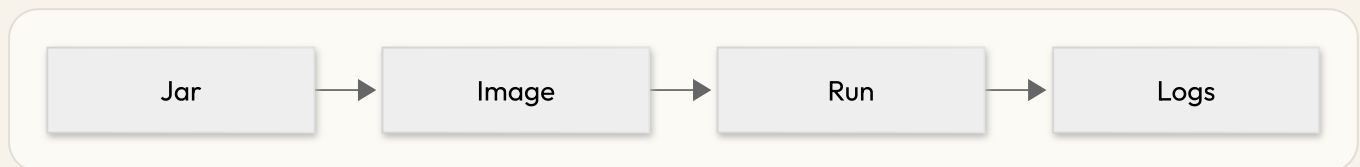
Ask. What is the difference between CloudWatch and CloudTrail?

Do. Write the alarm you would want on the diagnostics agent: error rate and token cost.

Docker, then the cluster

An image is the unit you ship. If the container does not run on your laptop, Kubernetes will not save it.

1. One process per container
2. Config by environment
3. A health URL
4. Non-root user



- Dockerfile: build the jar, copy it, expose the port, start the process.
- Do not bake secrets into the image.
- docker compose for Postgres and the app is the local rehearsal.

HEALTHCHECK or a /health route the orchestrator can call.

Miss. A container that works only because it uses your laptop's files.

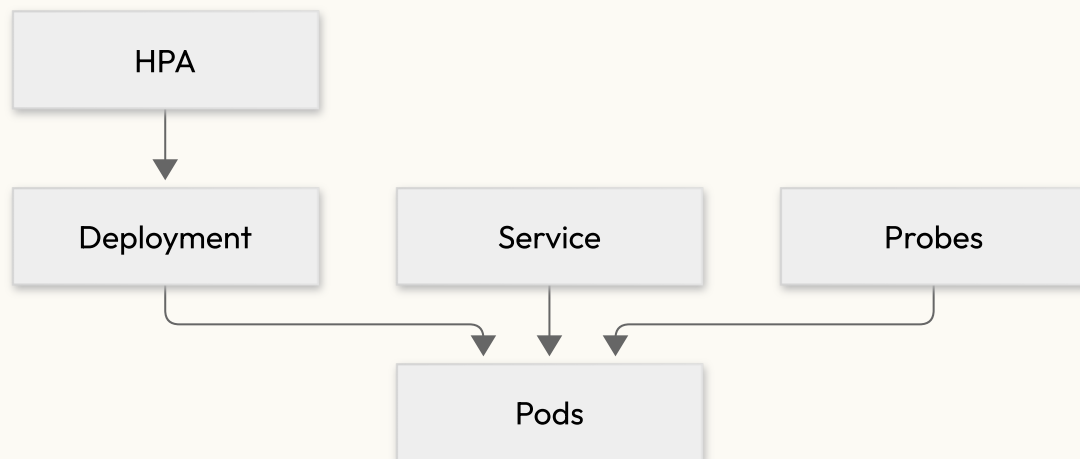
Ask. What is in the image, and what is not?

Do. Run the Spring service in Docker before you read another Kubernetes page.

Deployment, Service, probes, HPA

CKAD is a hands-on exam and it is relevant later. It is not how you learn. These objects are how you learn.

1. Deployment keeps N pods
2. Service is the stable address
3. Readiness is traffic
4. Liveness is restart



- Deployment declares the desired count and the image.
- Service selects pods by label. Ingress or a load balancer is the door from outside.
- Readiness false means leave me alone. Liveness false means restart me. Mixing them up causes outages.
- HPA scales on CPU or a custom metric. It does not fix a memory leak.

CKAD order: Docker, these objects, ConfigMap and Secret, RBAC, volumes, Helm, then the exam.

Miss. Using liveness to check a downstream dependency, so a database blip restarts every pod.

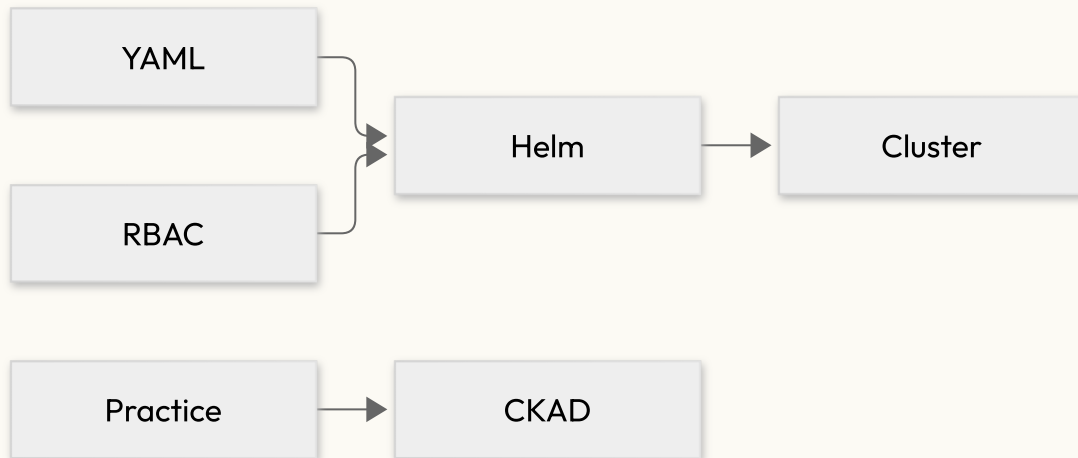
Ask. Why did this pod restart, and why did it stop receiving traffic?

Do. Write a Deployment and a Service for the FastAPI chunk service. Apply it locally if you have a cluster, or kind.

Config, RBAC, Helm, then CKAD

You already touch OpenShift at work. The public skill is the same objects with names you can explain, then Helm to stop copying YAML, then CKAD as proof.

1. ConfigMap for settings
2. Secret for credentials
3. RBAC is least privilege
4. CKAD after you can debug



- A ConfigMap changes the log level without a new image.
- A Secret is not encryption by itself. You still restrict who can get secrets.
- A Role that can only get and list pods in one namespace is the agent's read identity.
- Book CKAD when you can fix a CrashLoopBackOff without notes. Not this month.

`kubectll logs, describe, and get events. That is the debug loop.`

Miss. cluster-admin on the agent service account.

Ask. The pod is CrashLoopBackOff. What do you type first?

Do. Break a probe on purpose and fix it with describe.

Python through FastAPI

You do not need to become a Python developer. You need to read and write the agent service: types, tests, HTTP, and async where the HTTP library is async.

1. Functions and dicts
2. Type hints
3. One pytest
4. One POST endpoint



- venv and pip. Pin requirements. Do not install into the system Python.
- httpx or requests for the Java API. Pydantic for the body.
- SQLAlchemy only if this service owns tables. Otherwise call the Java service.
- asyncio matters when you wait on the model and the vector store together. Not before.

```
def retrieve(question: str) -> list[str]:  
    return index.search(question, k=3)
```

Miss. A 40-hour syntax course before a single endpoint exists.

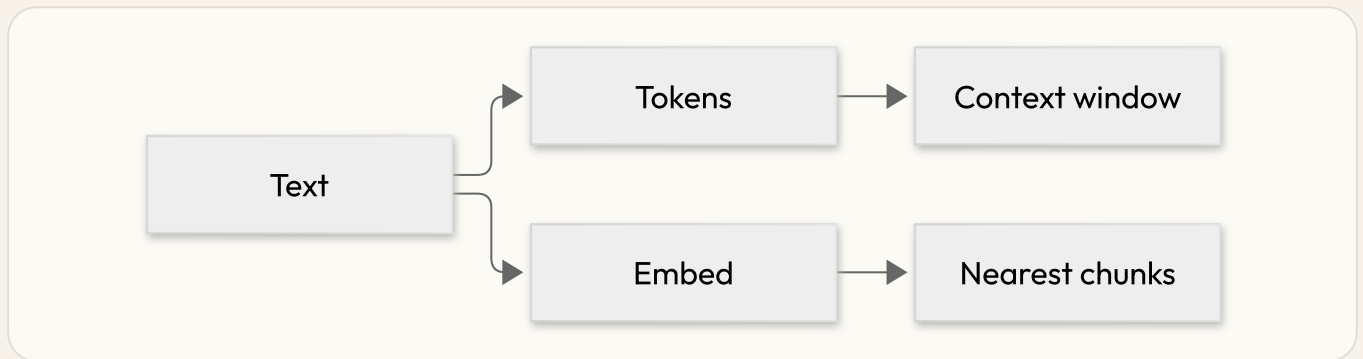
Ask. Show a typed function and a test.

Do. This week's Python is one file: read three markdown runbooks and return the overlapping lines.

Tokens, embeddings, retrieval

The engineering under the API. You should be able to say why a long log does not fit, and why similar vectors are not the same as a keyword match.

1. Tokens cost money and space
2. Temperature is randomness
3. Embeddings are coordinates
4. Retrieval is nearest chunks



- A context window is finite. You retrieve a few chunks instead of pasting the cluster.
- Temperature low for a root-cause draft. Higher only if you want variety, which you usually do not.
- Chunk on headings and size with overlap. Write down why you picked the size.

Structured output: ask for JSON with cause, evidence, action. Validate it. If it fails, retry once.

Miss. Turning the temperature up to make a diagnosis more creative.

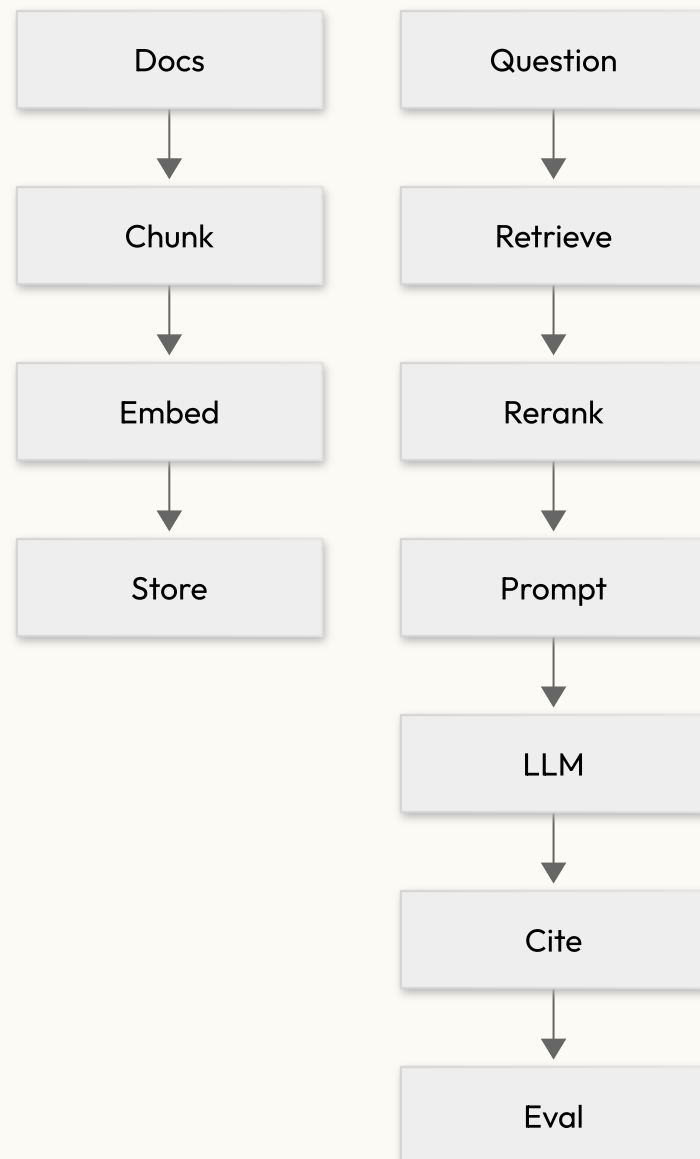
Ask. Why not send the whole log to the model?

Do. Count tokens on one synthetic log. Write the number in the README.

Production RAG

The pipeline is the product. A chat box without ingestion, citations, and an eval set is the project you were told not to build.

1. Ingest offline
2. Retrieve
3. Rerank if you need it
4. Cite
5. Score 20 questions



- Ingestion: clean text, chunk, embed, store with the source id.
- Query: embed the question, fetch top K, optionally rerank, build the prompt, call the model.
- The answer lists chunk ids. The eval set checks those facts.
- Streaming is a nicer UI. It does not fix a bad index.

If the index has no chunk about quota, the answer is I do not have that record.

Miss. Chat with PDF as the whole portfolio.

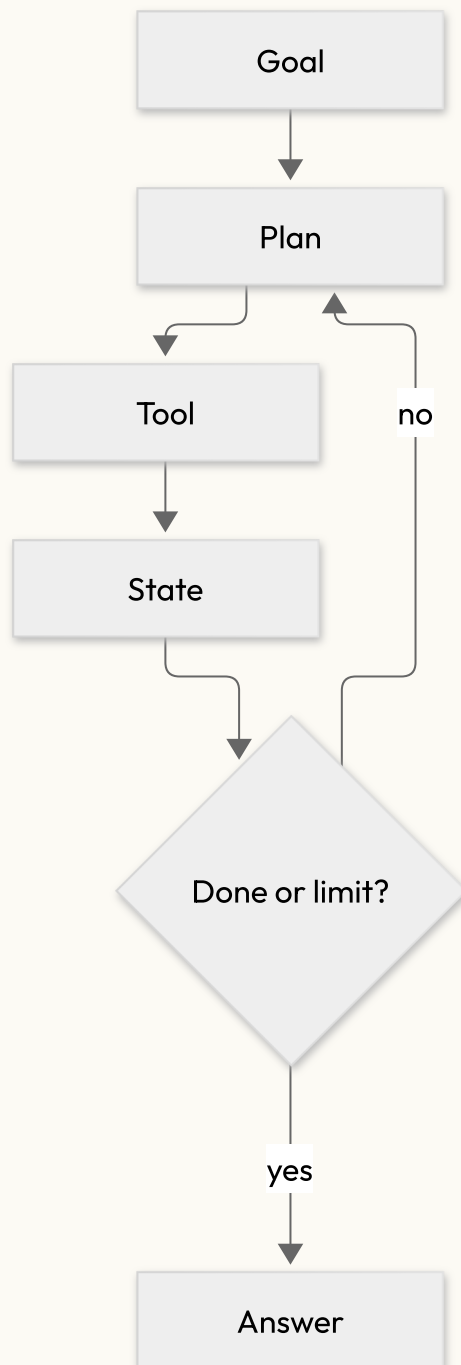
Ask. How do you measure whether retrieval got worse?

Do. Create the 20-question file before you pick a vector database.

Agents, MCP, evals, and safety

An agent is a loop with tools, state, and a stop condition. MCP is a way to expose tools. Neither removes the need for permissions, traces, and a test set.

1. Allow-list the tools
2. Cap the steps
3. Trace each call
4. Eval the task not the vibe



- State is the messages plus tool results. Memory across sessions is a database, not a hope.
- Timeouts, retries on reads, no retry on an unsafe write.
- Prompt injection: untrusted logs are data, not instructions. The system prompt says so, and the write tool is not available to that call.

- MCP lets a client discover tools. Your policy still decides which tools exist for this user.

Max 8 tool calls. Then the agent must answer or ask a human.

Miss. An agent with a shell tool on a cluster.

Ask. How do you make an agent safe enough for production?

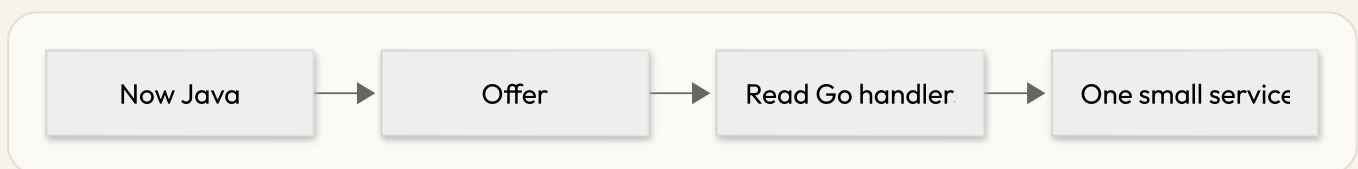
Do. List read tools and write tools for the storage platform. Only the second list needs approval.

GO · 20 MIN

Go after the offer, at reading depth

Apple storage and Cisco distributed roles mention Go beside Java. That is a reason to read Go, not a reason to pause Java. After the switch, one small service is enough.

1. Do not context-switch this winter
2. After April, read net/http
3. Goroutines are the concurrency primitive
4. Errors are values



- A handler, a struct, and an error return are the first page.
- You do not start with a framework tour.
- Storage-infrastructure posts want Linux, Kubernetes, and a language. Your storage stories plus Java already qualify you to talk. Go is the gap you close on the job.

```
func (s *Server) GetBucket(w http.ResponseWriter, r *http.Request)
```

Miss. Rewriting the flagship in Go in November.

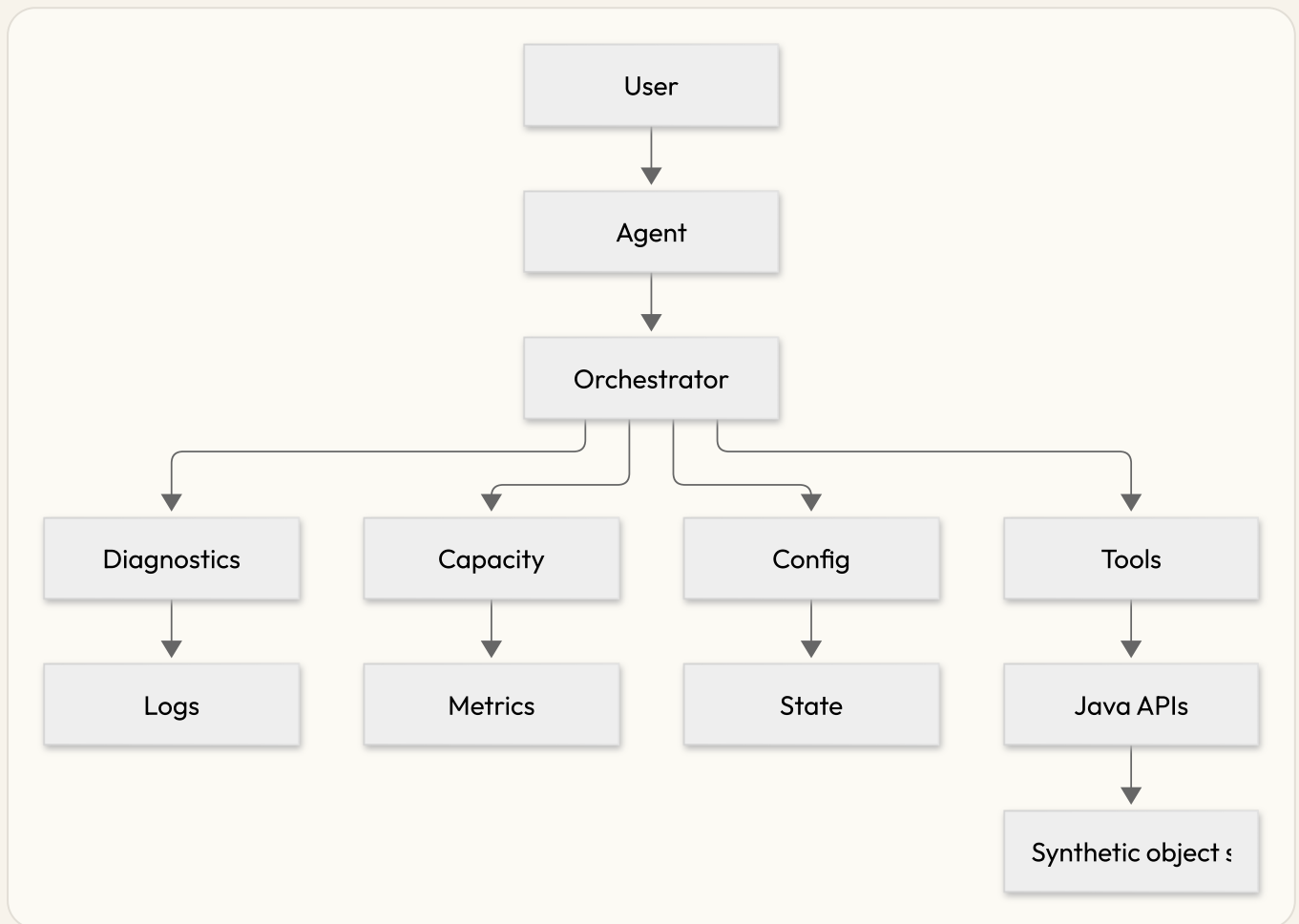
Ask. What will you learn in Go, and when?

Do. Add a single line to the April plan: read one Go HTTP file a week. Not before.

AI-native object storage operations

One project replaces the certificate pile. It is an operations console for a fake object store: Java services own metadata and events, a Python agent reads them, and a human approves anything that changes the cluster.

1. Synthetic data only
2. Java owns the system of record
3. Python owns the agent loop
4. Writes wait for a human



- Do not copy Dell code, schemas, hostnames, or customer data. Invent buckets, latency series, and runbooks.
- Services: metadata API in Spring Boot, event outbox, a worker, Postgres, Redis for the rate limit and session.
- The README has a diagram, how to run it, and an eval score.

Question the demo must answer: why is bucket latency high, using only the synthetic series.

Miss. A chat box with no tools and no score.

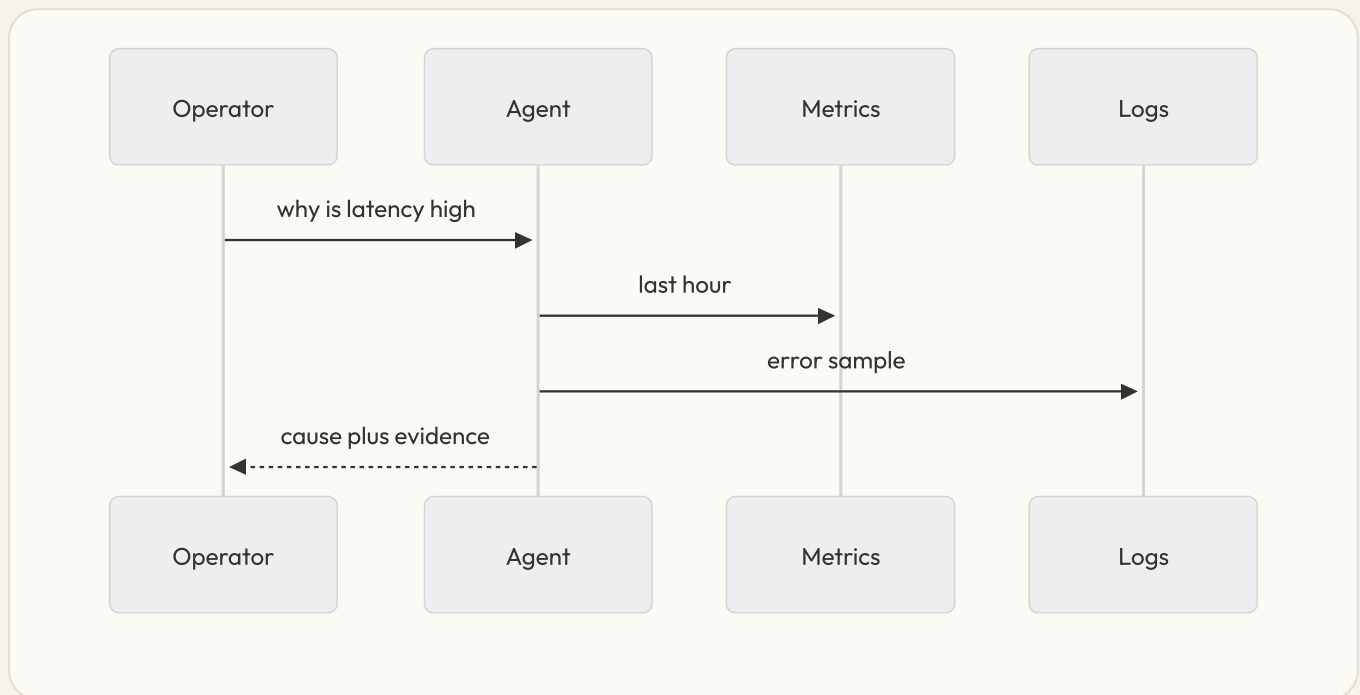
Ask. What can the agent do without a person, and what can it not?

Do. Create the repo and the architecture page. No model call yet.

Diagnostics agent

The first agent answers a latency question by reading metrics, logs, and traces, then naming a likely cause with the evidence attached.

1. Read tools only
2. Correlate time
3. Cite the series
4. Refuse if evidence is thin



- Tools: getLatency, getErrorRate, getRecentLogs. All read-only.
- The answer template: observation, likely cause, what would disprove it.
- Eval: ten scripted incidents with an expected cause label.

Cause: retry storm. Evidence: error rate up, then latency up, same minute.

Miss. A confident cause with no series id.

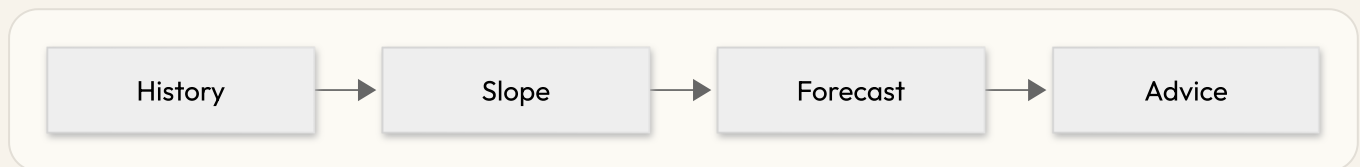
Ask. What would make you discard this hypothesis?

Do. Seed three fake incidents and write the expected labels.

Capacity agent

The second agent forecasts whether a cluster crosses a threshold. It is a query and a sentence, not a research model.

1. Use the stored series
2. State the assumption
3. Recommend, do not resize
4. Show the date it crosses



- A linear or seasonal baseline is enough. Say the assumption.
- The tool returns the series. The model writes the sentence. The number comes from your code, not from the model's imagination.
- If the series is shorter than two weeks, the agent says the forecast is weak.

At the current slope, 80 percent full on day 40. Assumption: no new tenant.

Miss. Letting the model invent the percentage.

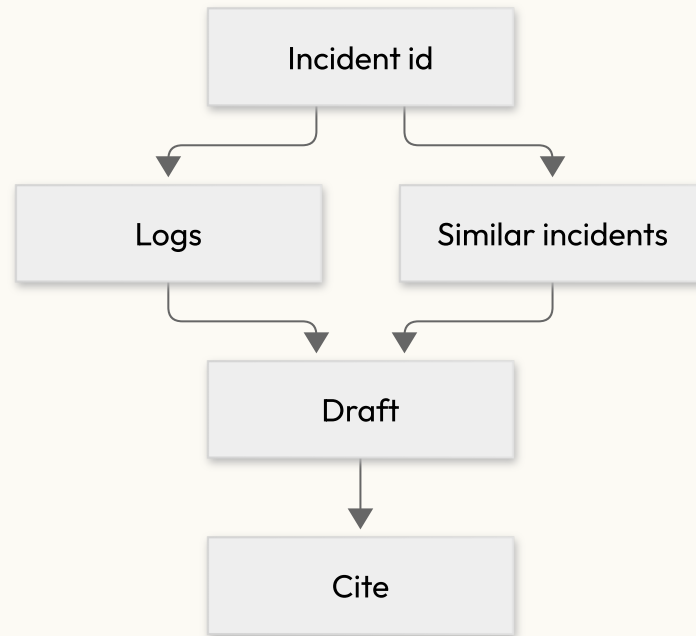
Ask. Where is the number computed?

Do. Implement the forecast in plain code. The model only explains it.

Incident RCA with retrieval

The third agent takes an incident id, pulls logs, and retrieves similar synthetic incidents and postmortems. That is RAG with a job to do.

1. Fetch by id
2. Retrieve similar text
3. Draft with citations
4. No fix is applied



- The corpus is synthetic postmortems you write. Twenty is enough.
- Similar means embedding or keyword overlap. Record which you used.
- The output is a draft for a human. It is not a merged pull request.

Similar to incident 12: disk saturation. Your logs show the same queue time.

Miss. Training a model on Dell tickets.

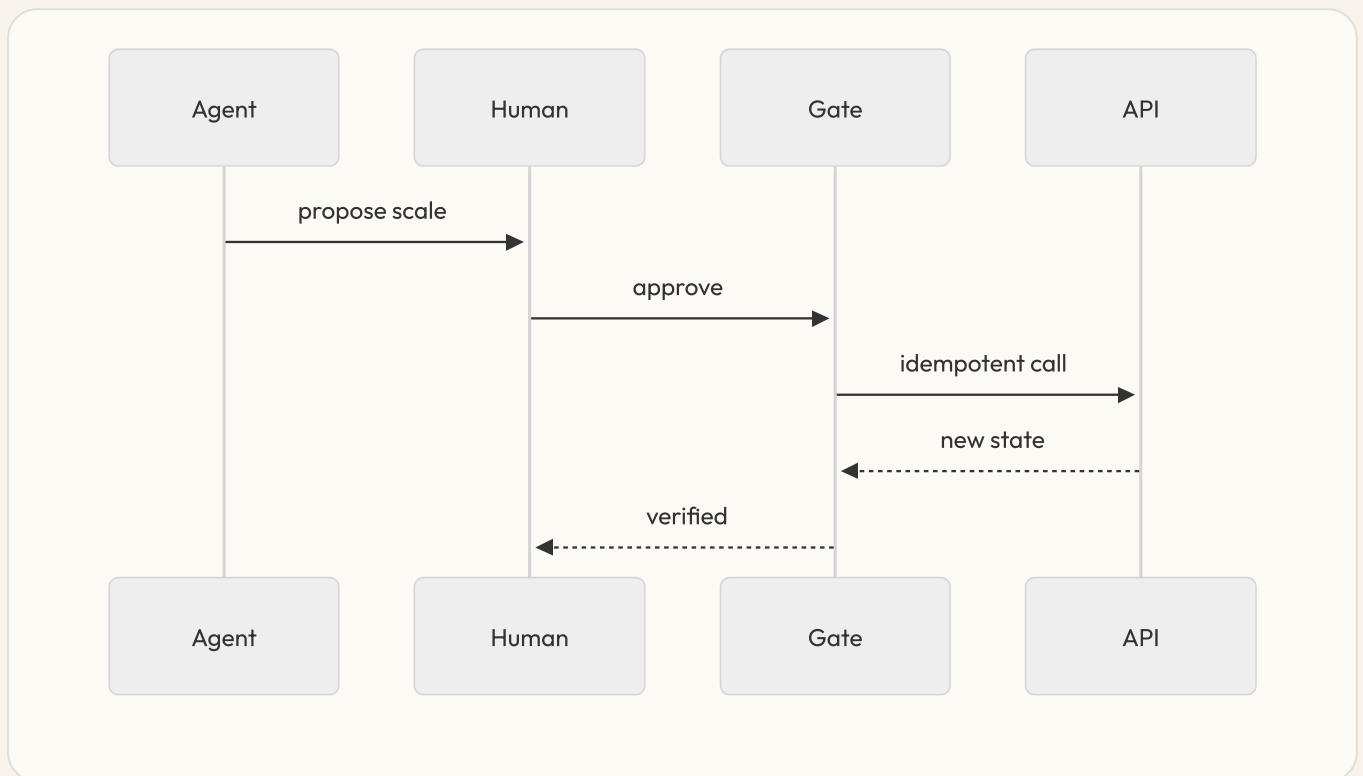
Ask. What is in the corpus, and what is forbidden to be in it?

Do. Write five fake postmortems. They are the index.

Safe executor

The fourth piece is the gate. The agent proposes. A person approves. A tool runs. A check verifies. The trace is the audit.

1. Proposal is data
2. Approval is a record
3. Tool is allow-listed
4. Verify after



- The proposal is a JSON document: action, target, reason, evidence ids.
- Approve stores the user id. The Java API checks that row before it mutates.
- Idempotency key so a double click does not scale twice.
- Verification reads the new state and attaches it to the trace.

POST /actions with Idempotency-Key. 200 on replay returns the same result.

Miss. The model calling kubectl with a string it composed.

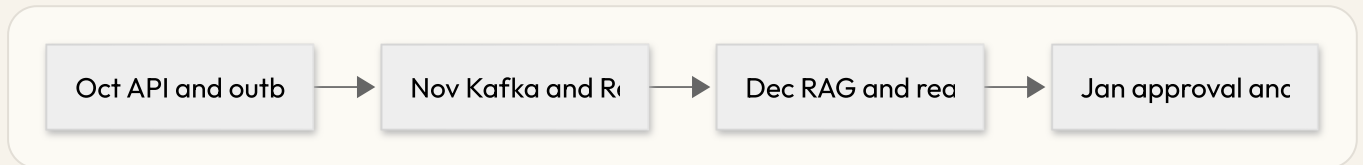
Ask. Who is allowed to approve, and where is that enforced?

Do. Implement the approval table in Java before the agent can ask for it.

Milestones through January

The project grows with the phases. It is not a December surprise.

1. October is the Java spine
2. November adds the bus and cache
3. December adds retrieval
4. January adds the gate and the score



- By 25 Oct: Spring service, Postgres, one idempotent POST, README run steps.
- By 25 Nov: outbox, a consumer, Redis rate limit, Docker compose.
- By 20 Dec: FastAPI retrieval over the fake postmortems, ten eval questions.
- By 15 Jan: approval gate, twenty eval questions, a number in the README, a diagram you can redraw.

Deploy to Kubernetes when compose is boring. AWS labels on the diagram can wait until the objects exist locally.

Miss. A Kubernetes cluster in October with no API.

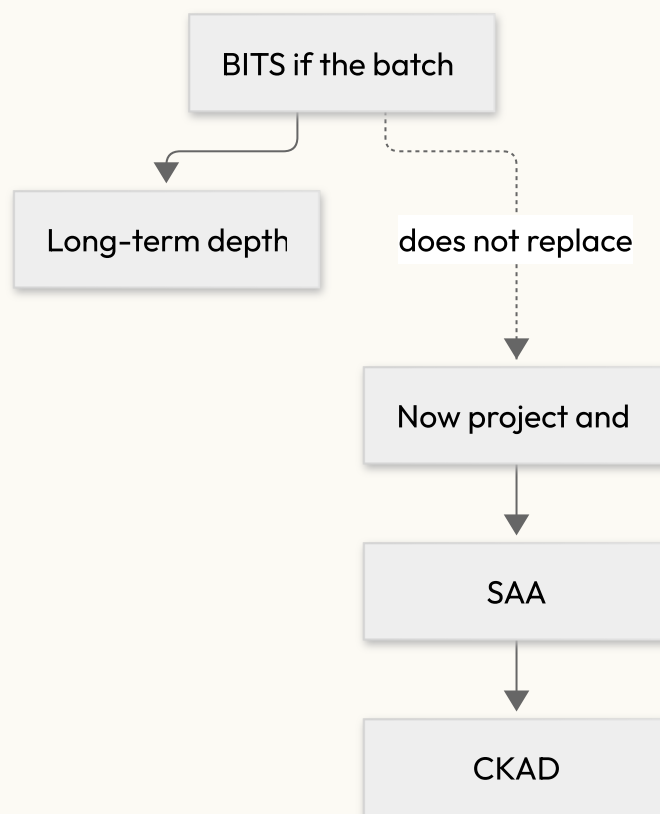
Ask. What is true in the repo on 25 November?

Do. Put the four dates in the README now, unchecked.

SAA, CKAD, and BITS on the side

Two credentials earn a place: AWS Solutions Architect Associate after you can draw the VPC, and CKAD after you can debug a pod. BITS M.Tech AI and ML is a degree for the next two years, including marriage and a job, not the thing that gets you the January interview.

1. SAA after a real diagram
2. CKAD after CrashLoop practice
3. AI-901 optional
4. BITS is parallel



- Skip Cloud Practitioner, AI-900, LangChain badges, and Python completion certificates.
- AI-901 replaced the English AI-900 exam in 2026. It is still optional.
- AWS Generative AI Developer Professional is the later GenAI credential, after Bedrock or an equivalent pipeline is something you operated.
- BITS WILP M.Tech AI and ML lists agentic systems, LLMs, cloud-native AI, and a dissertation. Public pages have quoted about ₹3.34 lakh across four semesters and have shown conflicting 2026 deadlines. It does not place you. If you join the October batch, confirm the date and the fee on the BITS site before you pay. It runs beside the job. It does not eat the 2.5 hours.

If the degree load collides with a mock week, the mock wins.

Miss. Enrolling and then skipping DSA because the degree feels like progress.

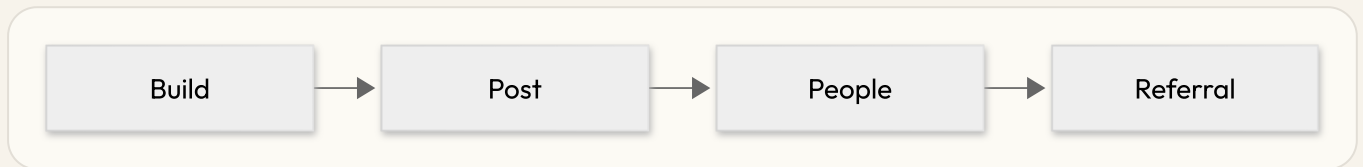
Ask. What does BITS not do for the April switch?

Do. Email or check the BITS page for the live deadline. Do it once, then decide.

One post, a short list of people

Networking is a weekly quota, not a persona. One technical post and a handful of real conversations beat a daily influencer schedule.

1. One post a week
2. Five engineers
3. Two recruiters
4. Two people at target companies



- Post from the work you did that week: an idempotent consumer, a probe you broke, a RAG eval number.
- No Dell internals. The synthetic platform is the public story.
- Ask for a referral only after a specific conversation, not as the first message.

This week I made a consumer safe to run twice. Here is the key, and here is the test.

Miss. Posting motivation quotes.

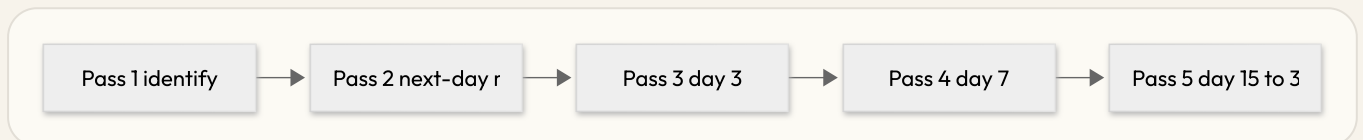
Ask. Who did you talk to, and about which system?

Do. Draft the first post from the outbox sketch. Four sentences.

Five passes, or it did not happen

You have forgotten solved problems because you never had to retrieve them. The pass is the study. The checklist's 342 rows are variations of about 40 shapes. Master the shape, then 120 to 150 representatives, not every row.

1. Name the pattern first
2. Close the video
3. Rewrite on an empty file
4. Red comes back sooner



- Pass 1: if you cannot name the pattern, read the lecture, then code.
- Pass 2 the next day with no video.
- The Patterns page stores the mark and the next date: day 1, 3, 7, 15, 30.
- Sliding window's many checklist titles are one loop. Expand, update, shrink, answer.

Green: solved alone. Yellow: hint. Red: solution. Blue: pattern yes, code no.

Miss. A tick mark the night you watched the editorial.

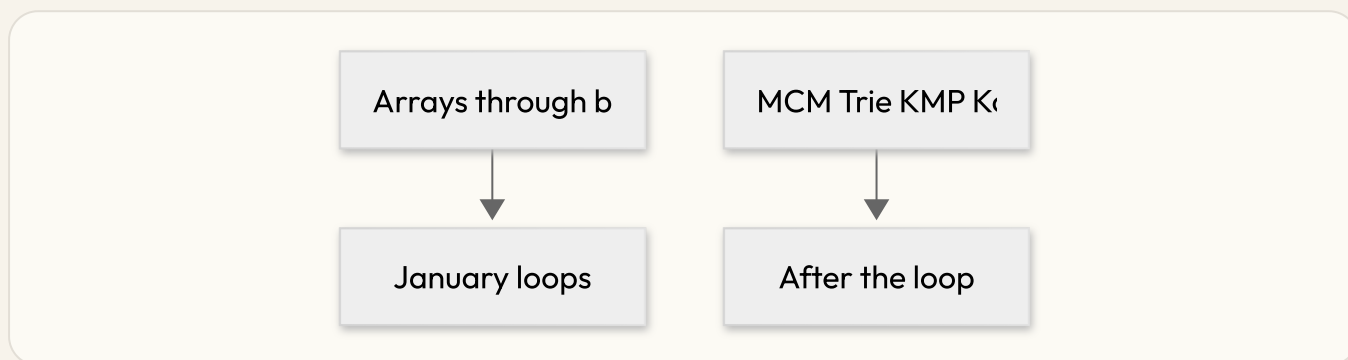
Ask. Which pass is this problem on?

Do. Pick one red problem and schedule only its rewrite tonight.

What January needs, and what waits

By January you can be handed a medium in the core set and start. Advanced string algorithms and exotic graph theorems are not equally important.

1. Core patterns are the job
2. One DP family done deeply
3. MCM is one problem if time
4. KMP is LPS once



- Must: arrays, hash, prefix sums, two pointers, sliding window, binary search, lists, stack, monotonic stack, intervals, greedy, trees, BST, heap, BFS/DFS, topo, Dijkstra, basic DP.
- Later: MCM, trie depth, KMP and Z as a specialty, Kosaraju, bridges, articulation points.
- The live pattern pages cover the must-set with Java templates.

Basic DP means stairs, robber, grid, subset, coins, one stock shape, one LIS, one LCS.
Not every variant.

Miss. Spending December on Kosaraju while house robber is shaky.

Ask. Name the pattern for a medium in under a minute.

Do. Open the drill and do ten prompts. The misses are the only new DSA.